

UNIVERSITÀ DEGLI STUDI DI MODENA E
REGGIO EMILIA

DIPARTIMENTO DI INGEGNERIA "ENZO FERRARI"

Laurea Triennale in Ingegneria Informatica

**Smart pot: prototipo di vaso intelligente
per la cura automatizzata delle piante**

Tutor:

Prof. Roberto Vezzani

Candidato:

Giulia Auri

Anno Accademico 2024/2025

Indice

1	Introduzione	5
1.1	Elementi significativi del progetto	6
1.2	Macrofunzionalità del sistema	6
1.3	Flusso d'interazione	8
2	Architettura Hardware	9
2.1	Microcontrollore	9
2.2	Sensore di umidità	13
2.2.1	Come funzionano i sensori di umidità?	13
2.2.2	Calibrazione del sensore	14
2.3	Relè	14
2.4	Pompa per l'irrigazione	15
2.5	Alimentazione	15
2.6	Schema del circuito	16
3	Architettura Software	18
3.1	Microcontrollore	19
3.2	Bridge Serial MQTT	23
3.2.1	Perchè si è scelto di utilizzare il protocollo MQTT?	27
3.3	Broker MQTT	28
3.4	Plant Descriptor	29
3.5	Data Collector	30
3.5.1	Main	31
3.5.2	Consumer	31
3.5.3	Producer	32
3.5.4	Factory	32
3.5.5	Policy Manager	33
3.6	File JSON	33
3.6.1	Json Manager	35
3.6.2	Perchè si è scelto di utilizzare il formato JSON?	36
3.6.3	Digital Twin e memorizzazione in Cloud	37
3.7	Backend	37
3.7.1	Perchè si è scelto di utilizzare il framework Flask?	38
3.8	Frontend	38

3.8.1	Tecnologie utilizzate	40
4	Sviluppi Futuri	42
4.1	Integrazione di sensori aggiuntivi	42
4.2	Notifiche push	42
4.3	Miglioramento delle animazioni e integrazione multi-vaso	42
4.4	Una migliore gestione delle policy	43
4.5	Algoritmi predittivi per l'irrigazione basati su machine learning	43
5	Conclusioni	44
5.1	Limiti	44
5.2	Potenzialità	44
5.3	Considerazioni finali	44
A	Codice del progetto	46
A.1	Bridge	46
A.1.1	bridge_Serial_MQTT.py	46
A.2	Plant Descriptor	50
A.2.1	plant_descriptor.py	50
A.3	Data Collector	50
A.3.1	data_collector_main.py	50
A.3.2	data_collector_consumer.py	52
A.3.3	data_collector_producer.py	55
A.3.4	factory.py	57
A.3.5	policy_manager.py	58
A.3.6	json_manager.py	62
A.4	Backend	67
A.4.1	web_api_server.py	67
A.5	Frontend	74
A.5.1	page.tsx	74
A.5.2	plant-dashboard.tsx	76

Elenco delle figure

1.1	Foto prototipo del <i>vaso intelligente</i>	5
1.2	Interfaccia utente della web application	7
1.3	Grafico che mostra l'andamento dell'umidità del terreno relativa	8
1.4	Schema generale del flusso delle interazioni	8
2.1	Arduino	10
2.2	Microcontrollore - Renesas RA4M1	10
2.3	Modulo ESP32-S3-MINI-1-N8	12
2.4	Switch per la comunicazione seriale	12
2.5	Sensore di umidità capacitivo	13
2.6	Relè	15
2.7	Pompa per l'irrigazione	15
2.8	Schema dei collegamenti	16
2.9	Foto collegamenti realizzati	16
3.1	Prima parte dello schema di comunicazione	18
3.2	Seconda parte dello schema di comunicazione	19
3.3	Sequence diagram	19
3.4	Smile animation	21
3.5	Drop animation	21
3.6	Sequence diagram relativo alla classe bridge	27
3.7	Class diagram che contiene le relazioni tra le classi Consumer, Pro- ducer, Factory, Json Manager e Policy Manager	31

Capitolo 1

Introduzione

L'oggetto di questo elaborato finale è la realizzazione di un *vaso intelligente* per piante, che faciliti e semplifichi la cura delle piante.

L'obiettivo principale è quello di ridurre gli errori nell'irrigazione e facilitarne la gestione quotidiana, esso è pensato per coloro che non hanno tempo o esperienza nella cura delle piante. Il vaso è dotato di un sensore di umidità del terreno e di una pompa d'irrigazione, che permettono di erogare acqua in modo automatico e controllato. Quando l'umidità del terreno scende sotto una soglia preimpostata, la pompa si attiva automaticamente e rilascia la giusta quantità di acqua nel vaso; invece, quando il terreno raggiunge un livello di umidità adeguato, l'irrigazione si interrompe. Oltre alla modalità automatica, il sistema prevede anche una modalità manuale, attivabile tramite un'applicazione web. In questo modo, l'utente può monitorare lo stato dei vasi e può attivare manualmente l'irrigazione da qualsiasi dispositivo mobile, garantendo maggiore flessibilità e controllo. La realizzazione del *vaso intelligente* si inserisce in un contesto di Smart home per la gestione automatizzata e da remoto per le piante d'appartamento.



Figura 1.1: Foto prototipo del *vaso intelligente*

1.1 Elementi significativi del progetto

L'aspetto più rilevante di questo lavoro non risiede unicamente nella realizzazione di un prototipo funzionante di *vaso intelligente*, ma nella progettazione di un'architettura scalabile, modulare ed estensibile. Il sistema non è stato concepito come una soluzione monolitica, bensì come un progetto articolato su più livelli:

- Il livello hardware, costituito dal microcontrollore e dai sensori/attuatori collegati, è responsabile della rilevazione dei dati ambientali e dell'attivazione della pompa di irrigazione.
- Il livello intermedio, rappresentato dal bridge seriale-MQTT, svolge la funzione di tradurre i dati provenienti da Arduino in messaggi scambiati con il broker secondo il paradigma publish/subscribe.
- Il livello cloud integra il Data Collector modulare per l'elaborazione dei dati, un backend sviluppato con Flask e un frontend realizzato con Next.js e React, che consente la visualizzazione e il controllo remoto del sistema.

La comunicazione tra microcontrollore e bridge avviene tramite connessione seriale, mentre lo scambio di informazioni tra bridge e cloud si basa sul protocollo MQTT. La scelta di questo protocollo garantisce un meccanismo di comunicazione leggero, asincrono e affidabile, particolarmente adatto a sistemi IoT distribuiti. Inoltre, l'adozione dell'architettura publish/subscribe permette di separare la fase di produzione dei dati da quella di consumo, favorendo la futura crescita del sistema e la sua integrazione con ulteriori dispositivi o servizi.

Un ulteriore elemento di innovazione è rappresentato dal Data Collector, organizzato in moduli indipendenti (producer, consumer, policy manager). Questa struttura consente di ampliare il sistema con nuove piante, sensori, attuatori o logiche decisionali senza dover intervenire sull'architettura di base. Allo stesso modo, anche il bridge e il codice eseguito sul microcontrollore sono stati progettati con un approccio modulare e flessibile: la loro configurazione permette infatti di integrare facilmente ulteriori dispositivi e funzionalità, garantendo la possibilità di evolvere il prototipo in un progetto più complesso e versatile.

Nel seguito dell'elaborato questi aspetti saranno descritti e motivati nel dettaglio, così da evidenziare, non solo la realizzazione di un prototipo funzionante, ma anche le scelte architettureali che lo rendono un contributo significativo nell'ambito dei sistemi IoT.

1.2 Macrofunzionalità del sistema

Questo prototipo consente di controllare la pianta da remoto attivando l'irrigazione con un semplice click sul pulsante play, controllare il livello dell'umidità in tempo reale e verificare quando è avvenuta l'ultima irrigazione.

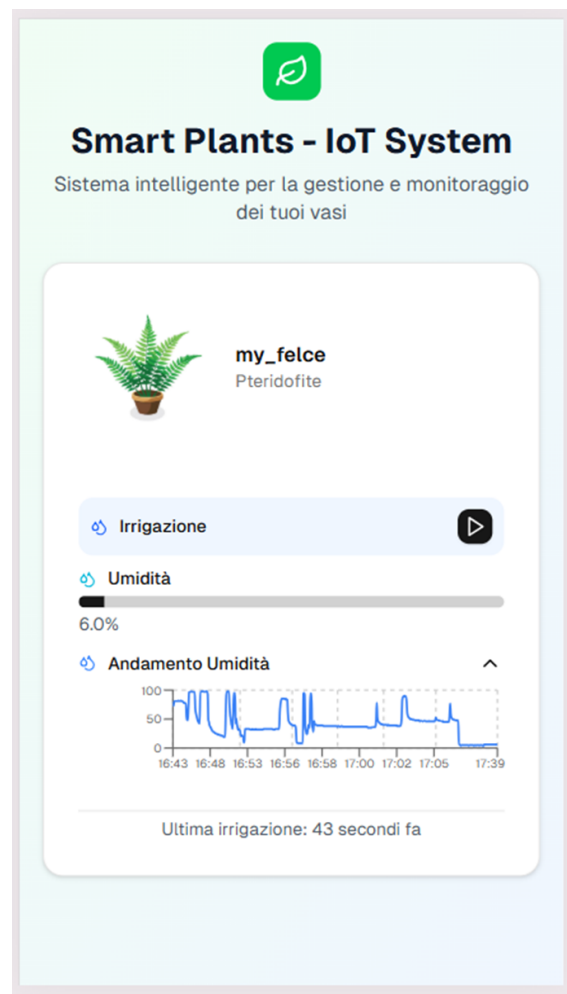


Figura 1.2: Interfaccia utente della web application

Inoltre si possono impostare regole/policy che in automatico regolano l'irrigazione. Le policy devono essere preventivamente caricate in un file json.

E' possibile raccogliere i dati dell'umidità del terreno in file json e memorizzare quando è stata accesa o spenta la pompa di irrigazione.

I dati storici dei valori registrati del sensore di umidità, sono visualizzabili anche dall'interfaccia web tramite un diagramma.



Figura 1.3: Grafico che mostra l'andamento dell'umidità del terreno relativa

1.3 Flusso d'interazione

Il funzionamento del prototipo è realizzato con la seguente catena di interazioni che consentono la raccolta dei dati e l'attivazione delle varie funzioni: i sensori e gli attuatori sono collegati alla scheda Arduino e al microcontrollore tramite un cavo. La scheda Arduino comunica con un PC locale in via seriale con cavo, ed elabora le informazioni e ne gestisce il flusso. Infine, il PC locale rende disponibili i dati in cloud, che sono fruibili all'utente grazie un'applicazione web. In questo modo l'utente potrà monitorare e controllare i vasi da qualsiasi dispositivo.

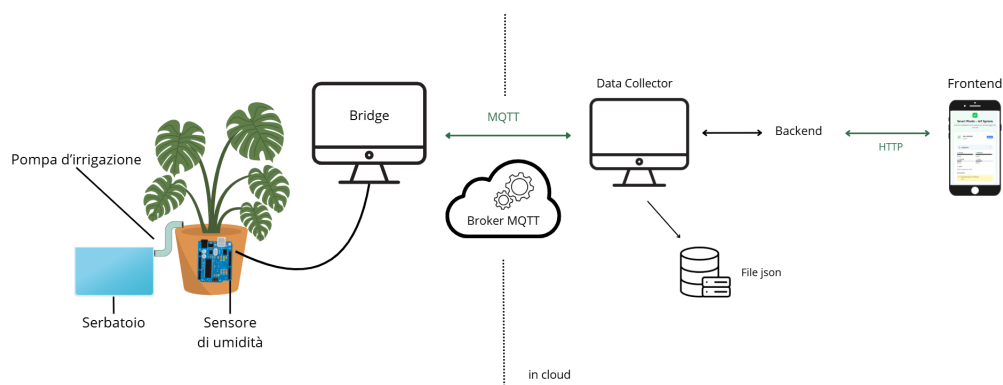


Figura 1.4: Schema generale del flusso delle interazioni

Capitolo 2

Architettura Hardware

Per la realizzazione di questo prototipo sono necessari i seguenti componenti:

- Arduino uno R4 wifi
- sensore di umidità
- relè per attivazione
- pompa d'irrigazione
- alimentazione per la pompa di irrigazione

2.1 Microcontrollore

Per questo prototipo si è scelto di utilizzare Arduino uno R4 wifi. Arduino è una scheda embedded open-source che ha riscosso grande popolarità grazie alla sua semplicità d'uso. La sua diffusione è da attribuire alla possibilità di avvicinare un pubblico più ampio alla programmazione di un microcontrollore, evitando l'acquisto di costose apparecchiature che richiedono operazioni di saldatura. Inoltre, mette a disposizione uno stack software efficiente che consente la programmazione con poche righe di codice. Grazie a queste caratteristiche, Arduino viene ampiamente utilizzato anche nella prototipazione rapida. Arduino è facilmente programmabile in C++ e utilizzando il Software Arduino (IDE).

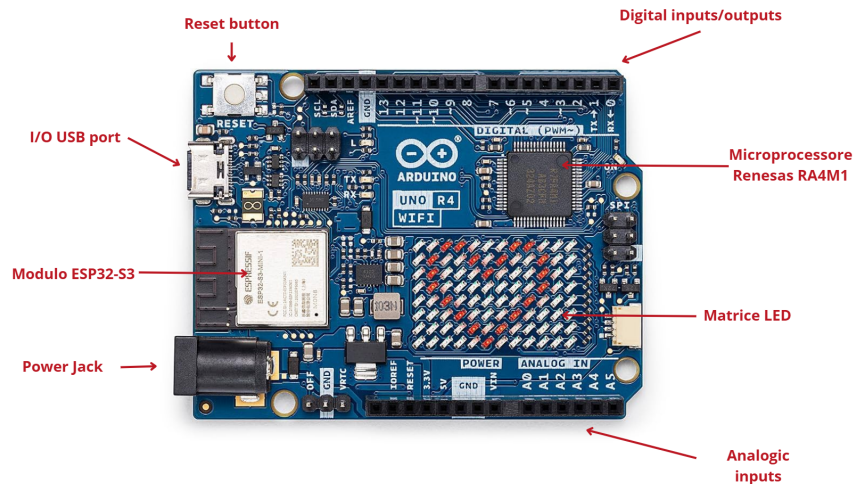


Figura 2.1: Arduino

Arduino uno r4 board comprende [1]:

- **Microcontrollore - Renesas RA4M1**

La scheda UNO R4 WiFi è dotata di un potente e robusto microcontrollore Renesas. I microcontrollori Renesas sono noti per le loro elevate prestazioni e robustezza, incluso il set di periferiche integrato.

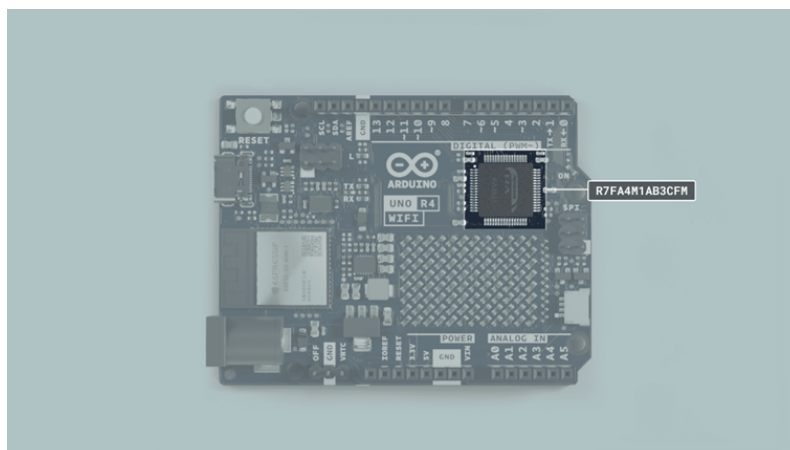


Figura 2.2: Microcontrollore - Renesas RA4M1

- **Memoria**

Le caratteristiche della scheda sono:

- 32 kB di SRAM
- Flash da 256 kB

- 8 kB di dati (EEPROM, nota anche come memoria "dati", essa è in grado di conservare i dati anche dopo lo spegnimento della scheda).

- **Pin**

I pin sono punti di ingresso/uscita per il collegamento attraverso cavi/bus ad altre componenti.

- Pin analogici: L'UNO R4 WiFi ha sei pin di ingresso analogici (A0-A5) che possono essere letti utilizzando la funzione `analogRead()`. La tensione di riferimento predefinita di questi pin è 5 V, anche se può essere modificata.
- Pin OPAMP: Il RA4M1 ha un OPAMP (amplificatore operazionale interno) interno esposto sull'UNO R4 WiFi.
- Pin digitali: La scheda UNO R4 WiFi è dotata di un totale di 14 pin digitali. I GPIO (General Purpose Input/Output) sono i pin di uso generico di un microcontrollore o di una scheda che puoi programmare liberamente per funzionare:
 - * in ingresso (Input): per leggere segnali digitali, ad esempio lo stato di un pulsante (alto/basso, acceso/spento).
 - * in uscita (Output): per pilotare dispositivi, ad esempio accendere un LED, attivare un relè o inviare segnali ad altri circuiti.

- **Matrice LED**

La matrice LED sulla scheda UNO R4 WiFi è disponibile per essere utilizzata nel programma, per visualizzare grafiche statiche o animazioni. Il pacchetto della scheda UNO R4 include la libreria `Arduino_LED_Matrix` per visualizzare i frame sulla matrice.

- **Porte seriali**

La scheda WiFi UNO R4 è dotata di 2 porte seriali hardware separate.

- Una porta è esposta tramite USB-C
- Uno è esposto tramite pin RX/TX

- **ESP32-S3-MINI-1-N8**

Di default, il modulo ESP32-S3 integrato nella UNO R4 WiFi funge da bridge seriale, gestendo la connessione al computer. Questo modulo gestisce anche il riavvio della MCU (MicroController Unit) principale, la Renesas RA4M1, quando necessario, ad esempio quando si riceve un nuovo sketch e si esegue un reset. L'ESP32 espone anche le sue linee dati, consentendo di programmarlo direttamente.

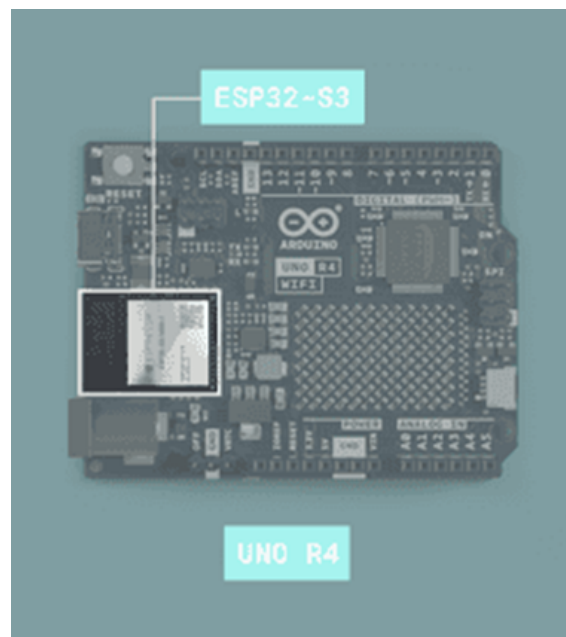


Figura 2.3: Modulo ESP32-S3-MINI-1-N8

Di default, l'ESP32 funge da bridge seriale tra un computer e il microcontrollore RA4M1. Le linee dati USB vengono instradate tramite switch, che per impostazione predefinita sono configurati per la comunicazione tramite il modulo ESP32.

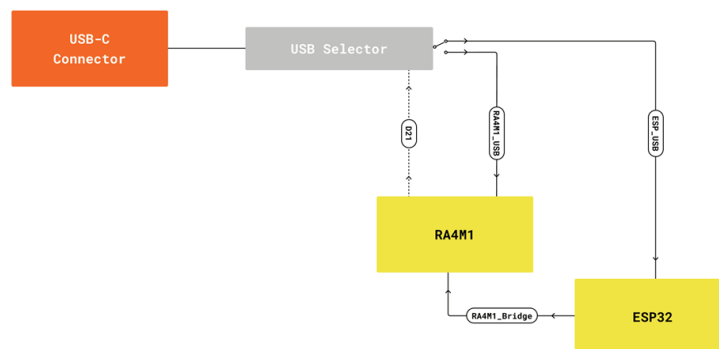


Figura 2.4: Switch per la comunicazione seriale

L'ESP32 integrato nella UNO R4 WiFi viene utilizzato per fornire funzionalità **Wi-Fi** alla scheda. Il modulo Wi-Fi ha un bitrate fino a 150 Mbps. Il modulo ESP32 è dotato di un'antenna di tracciamento integrata, il che significa che non è necessario averne una esterna per utilizzare le funzionalità di connettività della scheda.

Grazie al modulo ESP32, UNO R4 WiFi supporta **Bluetooth** LE e Bluetooth 5, con una velocità fino a 2 Mbps. Il modulo ESP32 è dotato di un'antenna integrata, che garantisce le funzionalità di connettività della scheda senza ulteriori antenne esterne aggiuntive.

2.2 Sensore di umidità

2.2.1 Come funzionano i sensori di umidità?

I sensori di umidità sono dei dispositivi in grado di misurare l'umidità presente nell'ambiente circostante. Considerando il tipo di grandezza misurata possono essere classificati in sensori di umidità assoluta (AH) e in sensori di umidità relativa (RH). Dal punto di vista elettrico, i sensori di umidità possono essere classificati in:

- **Sensori capacitivi**

Il principio su cui si basano i resistori capacitivi (progettati per misurare l'umidità relativa, o RH) è la variazione della permittività elettrica di un materiale dielettrico al variare dell'umidità. Da un punto di vista costruttivo, un sensore di umidità capacitivo è costituito da un condensatore, tra le sue armature viene inserito un opportuno materiale dielettrico, la cui costante dielettrica varia con l'umidità.

- **Sensori resistivi**

I Sensori resistivi, anziché la capacità, misurano la resistenza (impedenza) oppure la conduttività elettrica. Il principio su cui si basano questi sensori è la conduttività nei materiali conduttori non metallici dipendente dall'umidità (o, in altre parole, dal contenuto di acqua presente).

Per questo esempio si è scelto di utilizzare un sensore capacitivo come in figura.



Figura 2.5: Sensore di umidità capacitivo

2.2.2 Calibrazione del sensore

Nella realizzazione di questo prototipo, la calibrazione del sensore è stata effettuata tenendo conto di due condizioni di riferimento estreme:

- umidità massima: il sensore immerso in acqua;
- umidità minima: il sensore esposto all'aria secca.

Questi valori sono stati presi come il limite inferiore e superiore dell'intervallo di misura e i dati grezzi risultanti dal sensore sono stati poi normalizzati e portati a un intervallo del 0% e 100%, il cui aumento percentuale indica l'aumento di umidità nel terreno.

Condizione	Valore grezzo	Umidità (%)
Sensore immerso in acqua	50	100% (terreno saturo)
Sensore esposto all'aria	150	0% (terreno secco)

Tabella 2.1: Calibrazione del sensore di umidità del suolo

2.3 Relè

Un relè è un interruttore elettrico adatto al controllo di dispositivi e sistemi che utilizzano tensioni più elevate. Nel caso del modulo relè, il meccanismo è tipicamente un elettromagnete. Un relè può essere comandato da un segnale proveniente dal microcontrollore ed è costituito da due componenti:

1. La bobina di controllo che riceve un segnale.
2. Il contatto di potenza (COMmon, Normally Open, Normally Closed) che apre o chiude un circuito dove passa la corrente per un carico (in questo prototipo la pompa d'irrigazione).

Esistono principalmente due tipologie di relè:

- **Low-level trigger:** l'attivazione del relè avviene quando sul pin di ingresso (IN) viene applicato un livello logico basso (0V / GND). In questa condizione, il contatto *NO-COM* si chiude.
- **High-level trigger:** l'attivazione del relè avviene quando sul pin di ingresso (IN) viene applicato un livello logico alto (ad esempio 5V). In tal caso, il contatto *NO-COM* rimane aperto.

Nel caso specifico del modulo impiegato in questo progetto, il relè è di tipo **low-level trigger**, ovvero:

- Quando Arduino porta il pin di controllo a livello logico basso (LOW), il relè si attiva e la pompa viene alimentata.

- Quando Arduino porta il pin di controllo a livello logico alto (HIGH), il relè si disattiva e la pompa viene spenta.

Per questo prototipo si è scelto di utilizzare il modulo relè 5V low level trigger (come da immagine seguente).



Figura 2.6: Relè

2.4 Pompa per l'irrigazione

Per la realizzazione del prototipo è stata adottata una mini pompa ad acqua a corrente continua, con tensione nominale di alimentazione pari a 3 V o 4,5 V, la cui portata a vuoto è di circa 100 L/h. Si tratta di una pompa ad immersione, progettata per funzionare direttamente all'interno del serbatoio.



Figura 2.7: Pompa per l'irrigazione

2.5 Alimentazione

I pin di alimentazione forniti sulla scheda Arduino sono progettati per carichi ridotti e non devono essere utilizzati per alimentare dispositivi ad alto assorbimento, per evitare malfunzionamenti o danni alla scheda [1]. Di conseguenza per alimentare la

pompa d'irrigazione senza sovraccaricare Arduino, è stata prevista un'alimentazione esterna in corrente continua (DC) da 3 V. Nel prototipo, questa alimentazione è stata realizzata mediante un porta-pile contenente due batterie da 1,5 V ciascuna.

2.6 Schema del circuito

Il circuito del prototipo è stato progettato per permettere il corretto funzionamento di sensori, attuatori e microcontrollore. Di seguito vengono descritti i collegamenti principali tra i dispositivi elettronici del prototipo.

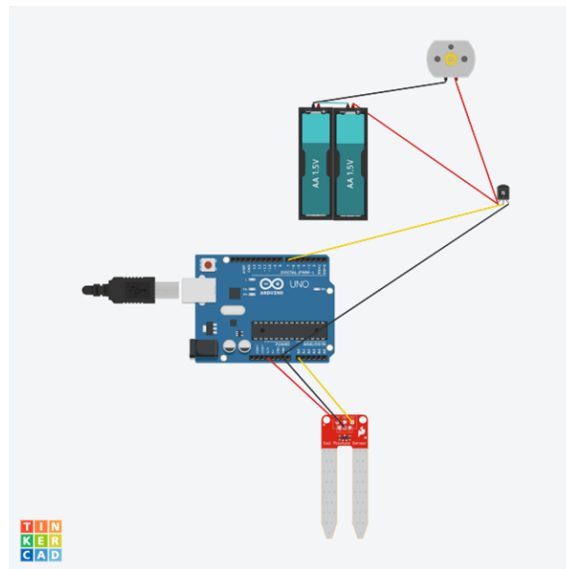


Figura 2.8: Schema dei collegamenti



Figura 2.9: Foto collegamenti realizzati

Componente	Terminale / Pin	Collegamento a
Sensore capacitivo umidità	VCC	5V di Arduino
	GND	GND Arduino
	A0 (uscita analogica)	A0 di Arduino
Relè (low-level trigger)	VCC	5V Arduino
	GND	GND Arduino
	IN (controllo)	Pin digitale, es. D7 Arduino
Pompa di irrigazione (3V)	+ (rosso)	NO (Normally Open) del relè
	– (nero)	– dell'alimentatore 3V
Alimentatore pompa	+ (3V)	COM del relè
	– (3V)	– pompa (nero) comune Arduino-Relè

Tabella 2.2: Tabella dei collegamenti principali elettronici

Capitolo 3

Architettura Software

Il microcontrollore è collegato direttamente al bridge con una connessione seriale. Il bridge si occupa di inviare/ricevere i dati ad Arduino e di inviare/ricevere i dati via MQTT dal Cloud.

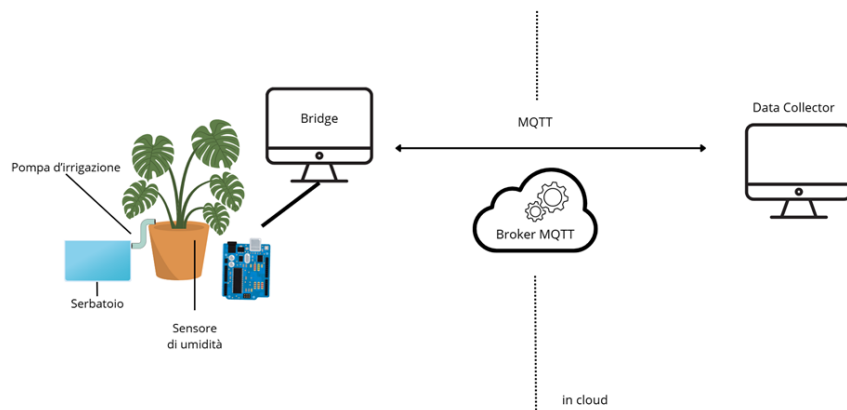


Figura 3.1: Prima parte dello schema di comunicazione

Successivamente il data collector salva i dati nei file json, il backend li legge e, attraverso il protocollo HTTP, li invia al frontend.

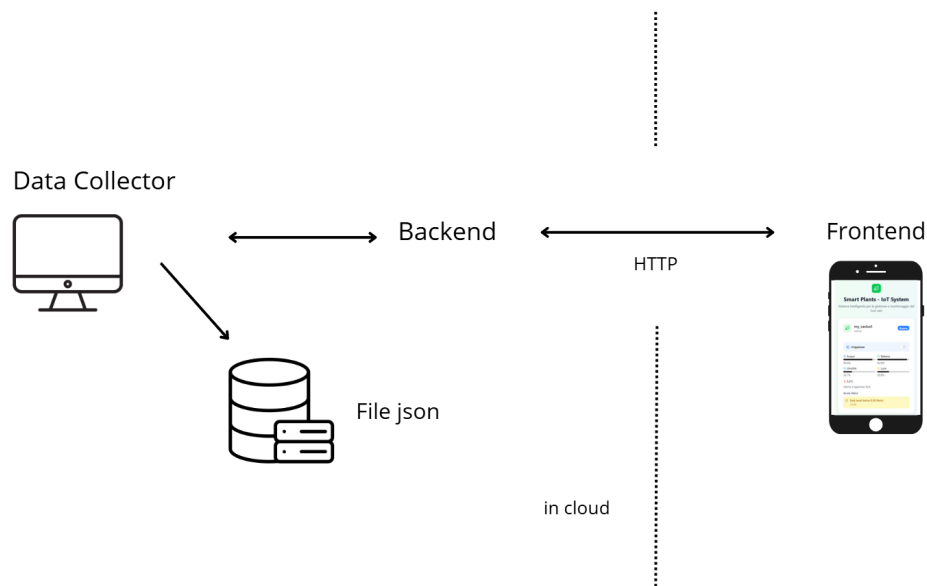


Figura 3.2: Seconda parte dello schema di comunicazione

Per illustrare in modo esauriente il flusso di comunicazione tra il microcontrollore, il bridge e il data collector si rimanda al sequence diagram sotto riportato.

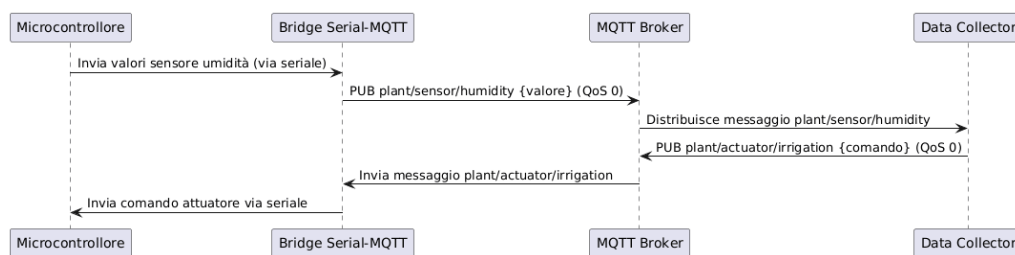


Figura 3.3: Sequence diagram

In seguito andremo ad analizzare ogni singolo elemento.

3.1 Microcontrollore

Per programmare il microcontrollore di Arduino, lo stesso è stato collegato al PC tramite cavo USB ed è stato caricato il codice del programma in linguaggio C++ utilizzando l'interfaccia Arduino IDE.

Questo programma integra tre funzioni principali:

- Raccolta dei dati dal sensore di umidità del terreno.
- Comunicazione seriale con il bridge, tramite pacchetti con una struttura definita.

- Controllo dell'attuatore (la pompa di irrigazione) attraverso un relè e visualizzazione di feedback visivo mediante la matrice LED integrata.

All'inizio del codice, viene inclusa la libreria `Arduino_LED_Matrix`, fondamentale per gestire la matrice LED sulla scheda, che informa l'utente sullo stato della pianta. È importante definire i seguenti elementi:

- una figura statica (uno smile) appare quando il sistema è acceso ma l'irrigazione è inattiva;
- una breve animazione (`animation_drop`) simula la caduta delle gocce d'acqua, utile come feedback quando la pompa è in funzione.

Si vanno ora a descrivere i pin di collegamento utilizzati in questo prototipo:

- `SENSOR_PIN` (A0) per il sensore analogico di umidità;
- `RELAY_PIN` (7) per il controllo del modulo relè;

Due costanti (`SENSOR_TYPE`, `ACTUATOR_TYPE`) identificano univocamente i vari tipi di sensori e attuatori collegati alla board nei messaggi scambiati con il bridge.

Il codice dell'IDE Arduino svolge due funzioni principali:

1. `void setup()`: in questo metodo si inizializzano le variabili, si dichiarano le librerie necessarie, si impostano le comunicazioni seriali e si definisce lo stato dei pin. Questa funzione è la prima a essere eseguita ogni volta che accendiamo o resettiamo Arduino. All'interno di questa funzione:
 - la comunicazione seriale viene avviata a 9600 baud;
 - il pin del relè viene configurato come uscita digitale, impostandolo su stato alto (relè disattivato, dato che il modulo funziona con logica inversa low-level trigger);
 - la matrice LED viene avviata, caricando come frame iniziale un'immagine statica di uno smile.
2. `void loop()`: esegue ciclicamente le operazioni definite al suo interno e interagisce con la scheda Arduino. In questo caso, ogni secondo viene effettuata una lettura analogica dal sensore di umidità. Il valore, che varia tra 0 e 1023, viene rimappato nell'intervallo 0–253 e inviato tramite porta seriale seguendo questo protocollo:

```
0xFF <ID sensore> <Tipo sensore> <Valore> 0xFE
```

Dove:

- 0xFF e 0xFE segnano rispettivamente l'inizio e la fine del pacchetto;
- <ID sensore> è un identificatore numerico (in questo caso 1);

- <Tipo sensore> in questo caso è la lettera 'H' (Umidità), poichè abbiamo un unico sensore di umidità;
- <Valore> è il dato acquisito, opportunamente normalizzato.

Nella funzione `void loop()`, il microcontrollore è sempre in ascolto di messaggi che arrivano dal bridge. Se un comando è disponibile, viene interpretato e, se è destinato all'attuatore di tipo 'I' (Irrigation), si decide lo stato del relè che può essere:

- comando 'A' → attivazione della pompa (uscita impostata a livello basso, relè attivo);
- comando 'S' → disattivazione della pompa (uscita a livello alto, relè disattivo).

Grazie a ciò, il codice è flessibile e potrà essere facilmente aggiornato in seguito per includere nuovi attuatori e sensori.

Per rendere l'interfaccia più user-friendly e fornire un feedback immediato sullo stato del sistema, si utilizza una matrice LED come interfaccia grafica semplice, che presenterà i seguenti stati:

- se il relè è spento, appare un smile statico;

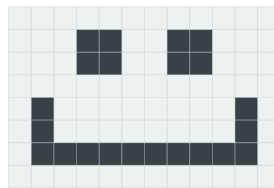


Figura 3.4: Smile animation

- se il relè è acceso, viene mostrata un'animazione della goccia che si ripete, con un intervallo di 100 ms tra i frame.

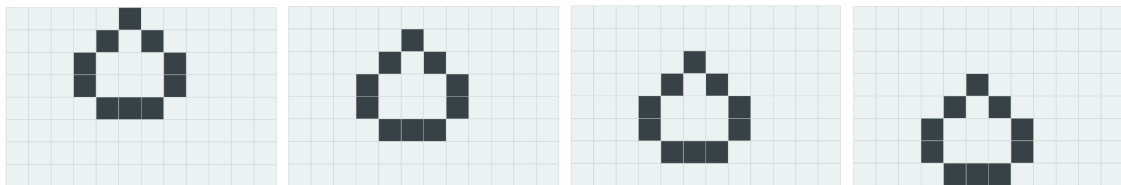


Figura 3.5: Drop animation

```

1 #include "Arduino_LED_Matrix.h"
2 ArduinoLEDMatrix matrix;
3 const uint32_t smile[8] = {
4     0x19819,
5     0x80004024,

```

```

6   0x27fe000,
7   0x0,
8   0x0,
9   0x0,
10  0x0,
11  0x0
12 };
13
14 const uint32_t animation_drop[][8] = {
15     {0x400a011, 0x1100e00, 0x0, 66, 0, 0, 0, 0},
16     {0x400a,    0x1101100, 0xe0000000, 66, 0, 0, 0, 0},
17     {0x4,       0xa01101, 0x100e0000, 66, 0, 0, 0, 0},
18     {0x0,       0x400a01, 0x101100e0, 66, 0, 0, 0, 0}
19 };
20 const int NUM_FRAMES = sizeof(animation_drop) / sizeof(animation_drop[0]);
21
22
23 const int SENSOR_PIN = A0;
24 const char SENSOR_TYPE = 'H';
25 unsigned long timestamp;
26 const int RELAY_PIN=7;
27 const char ACTUATOR_TYPE = 'I';
28
29 void setup() {
30
31     Serial.begin(9600);
32     timestamp = millis();
33     pinMode(RELAY_PIN, OUTPUT);
34     digitalWrite(RELAY_PIN, HIGH); // relay off at startup (low-level trigger
35     )
36
37     // LED Matrix
38     matrix.begin();
39     matrix.loadFrame(smile);
40 }
41
42 void loop() {
43     int val;
44     if (millis() - timestamp > 1000){
45         // sensor
46         val = analogRead(SENSOR_PIN);
47
48         // data packet
49         // FF number type value FE
50         Serial.write(0xFF);
51         Serial.write(1);
52         Serial.write(SENSOR_TYPE);
53
54         Serial.write(map(val, 0, 1023, 0, 253));
55
56         Serial.write(0xFE);
57
58         // actuator
59         if (Serial.available() > 0)
60         { int val;
61           char type;
62           type = Serial.read();
63           val = Serial.read();
64           if (type==ACTUATOR_TYPE) {
65               if (val=='A') digitalWrite(RELAY_PIN, LOW); //accesso
66               if (val=='S') digitalWrite(RELAY_PIN, HIGH); //spento
67           }
68     }

```

```
69     timestamp = millis();
70
71
72 }
73 // --- LED Matrix Management ---
74 if (digitalRead(RELAY_PIN) == LOW) {
75     // relay on -> animation drop
76     for (int i = 0; i < NUM_FRAMES; i++) {
77         matrix.loadFrame(animation_drop[i]);
78         delay(100);
79     }
80 } else {
81     // relay off -> smile
82     matrix.loadFrame(smile);
83 }
84 }
```

3.2 Bridge Serial MQTT

La classe Bridge, eseguita localmente, funge da collegamento tra il microcontrollore Arduino e il sistema software, le sue funzioni principali sono:

- **Comunicazione seriale:** gestisce lo scambio di dati, in entrambe le direzioni, con la scheda Arduino tramite una connessione cablata, in particolare riceve i valori dai sensori ed invia i comandi agli attuatori.
- **Comunicazione tramite MQTT:** pubblica/riceve dati sui topic dal broker MQTT. I topic utilizzati sono:
 - plant/sensor/humidity per inviare i dati dei sensori;
 - plant/actuator/irrigation per ricevere i comandi di irrigazione.

All'interno della classe sono definite anche alcune costanti che aiutano a mantenere il codice leggibile ed estensibile, esse sono:

- **SENSOR_MAPPING:** associa a ciascun identificativo di sensore (H, T, L, B) un nome descrittivo (umidità, temperatura, luminosità, livello batteria).
- **ACTUATOR_MAPPING:** mappa l'identificativo degli attuatori con il nome degli attuatori corrispondenti (nel prototipo 0 corrisponde all'irrigatore).
- **ACTUATOR_CHAR_MAPPING:** mappa l'identificativo degli attuatori con i rispettivi caratteri seriali compatibilmente a ciò che viene fatto nel microcontrollore (nel prototipo 0 corrisponde b'0').

Queste mappature permettono di convertire i dati grezzi, ricevuti da Arduino, in informazioni chiare e compatibili con il bridge, anche nel caso in cui siano collegati più sensori o attuatori allo stesso microcontrollore.

Si procede ora all'analisi dettagliata dei principali metodi di questa classe.

Il metodo `setupSerial(self)` si occupa di configurare la porta seriale, necessaria per comunicare con Arduino. In particolare, identifica la porta disponibile, in base alla descrizione o al nome, apre la connessione a una velocità di 9600 baud, inizializza un buffer interno per raccogliere i dati e gestisce eventuali errori di connessione.

```

1 def setupSerial(self):
2     self.ser = None
3     self.portname = None
4
5     if self.config.get("Serial","UseDescription", fallback=False):
6         self.portname = self.config.get("Serial","PortName", fallback="COM5")
7     else:
8         print("list of available ports: ")
9         ports = serial.tools.list_ports.comports()
10
11        for port in ports:
12            print (port.device)
13            print (port.description)
14            if self.config.get("Serial","PortDescription", fallback="arduino").
lower() \
15                in port.description.lower():
16                self.portname = port.device
17
18        if self.portname is None:
19            self.portname = self.config.get("Serial","PortName", fallback="COM5")
20            print(f"Usando porta configurata: {self.portname}")
21
22        try:
23            if self.portname is not None:
24                print ("connecting to " + self.portname)
25                self.ser = serial.Serial(self.portname, 9600, timeout=0)
26                print(f"Connesso alla porta {self.portname}")
27            else:
28                print("Nessuna porta trovata")
29        except Exception as e:
30            print(f"Errore connessione seriale: {e}")
31            self.ser = None
32
33        # self.ser.open()
34        # internal input buffer from serial
35        self.inbuffer = []

```

Il metodo `setupMQTT(self)` ha il compito di stabilire la connessione con il broker MQTT, che in questo caso è locale e utilizza la porta 7883. A questo punto si crea il client MQTT, si impostano le callback per gestire gli eventi, si avvia il loop di comunicazione in background e si sottoscrive al topic `plant/actuator/irrigation`, relativo ai comandi di irrigazione.

```

1 def setupMQTT(self):
2     self.clientMQTT = mqtt.Client()
3     self.clientMQTT.on_connect = self.on_connect
4     self.clientMQTT.on_message = self.on_message
5     print("connecting to MQTT broker...")
6     self.clientMQTT.connect(
7         self.config.get("MQTT","Server", fallback= "localhost"),
8         self.config.getint("MQTT","Port", fallback= 7883),
9         60)
10

```

```
11 self.clientMQTT.loop_start()
```

Il metodo `on_connect(self, client, userdata, flags, rc)` è la callback che viene eseguita quando il client MQTT si connette al broker. Questo metodo registra nel log l'esito della connessione, si sottoscrive al topic dei comandi e, in caso di riconnessione, rinnova automaticamente le sottoscrizioni. La sottoscrizione ai dati sensoriali, come nel caso del topic `plant/sensor/humidity`, avviene con QoS pari a 0: dato l'alto tasso di aggiornamento, la perdita di singoli messaggi non è un problema rilevante.

```
1 def on_connect(self, client, userdata, flags, rc):
2     print("Connected with result code " + str(rc))
3
4     for actuator_id, actuator_name in ACTUATOR_MAPPING.items():
5         topic = MqttConfigurationParameters.build_command_plant_topic(
6             actuator_name)
7         self.clientMQTT.subscribe(topic)
8         print(f"Subscribed to: {topic}")
```

Il metodo `on_message(self, client, userdata, msg)` viene eseguito ogni volta che si riceve un messaggio MQTT. Questo metodo decodifica il payload e lo traduce in un comando interpretabile, mappa i comandi ricevuti, invia il comando corrispondente ad Arduino tramite la connessione seriale e gestisce eventuali comandi non riconosciuti. Un esempio tipico di messaggio ricevuto è rappresentato da:

```
1 topic: plants/commands/irrigation
2 payload: start
```

Il payload viene interpretato e tradotto in un pacchetto seriale destinato ad Arduino, che ha la seguente struttura: `<nome_attuatore> <azione>`. Ad esempio nel prototipo:

- 'I' 'A' → attiva il relè (pompa ON),
- 'I' 'S' → disattiva il relè (pompa OFF).

```
1 def on_message(self, client, userdata, msg):
2     print(f"Comando ricevuto - Topic: {msg.topic}, Payload: {msg.payload}")
3
4     topic_parts = msg.topic.split('/')
5     actuator_type = topic_parts[-1] # "irrigation", "lighting", etc.
6
7     actuator_id = None
8     for id, name in ACTUATOR_MAPPING.items():
9         if name == actuator_type:
10             actuator_id = id
11             break
12
```

```

13     if actuator_id is None:
14         print(f"Attuatore non riconosciuto: {actuator_type}")
15         return
16
17     actuator_char = ACTUATOR_CHAR_MAPPING.get(actuator_id)
18     if not actuator_char:
19         print(f"Carattere seriale non trovato per attuatore {actuator_id}")
20         return
21
22     command = msg.payload.decode('utf-8').lower()
23
24     if self.ser is not None:
25         if command == "start" or command == "on" or command == "1":
26             self.ser.write(actuator_char) # Tipo attuatore
27             self.ser.write(b'A') # Comando ATTIVA
28             print(f"Comando ATTIVA inviato per {actuator_type}")
29         elif command == "stop" or command == "off" or command == "0":
30             self.ser.write(actuator_char) # Tipo attuatore
31             self.ser.write(b'S') # Comando DISATTIVA
32             print(f"Comando DISATTIVA inviato per {actuator_type}")
33         else:
34             print(f"Comando non riconosciuto: {command}")
35     else:
36         print("Serial port not available!")

```

Infine, il metodo `loop(self)` costituisce il ciclo principale del sistema, responsabile della comunicazione continua con Arduino. Durante ogni iterazione, il metodo legge i dati provenienti dalla connessione seriale e li accumula in un buffer fino al carattere di terminazione (0xFE). Una volta rilevato il pacchetto completo, viene richiamata la funzione `useData()`, che ha il compito di interpretare il contenuto del buffer, estrarre i valori dei sensori secondo la struttura del messaggio e pubblicarli tramite MQTT sui topic corrispondenti. Infine, il buffer viene svuotato per prepararsi alla lettura del pacchetto successivo.

```

1 def loop(self):
2     while (True):
3         if not self.ser is None:
4             if self.ser.in_waiting>0:
5                 lastchar=self.ser.read(1)
6                 print(f"Byte ricevuto da Arduino: {lastchar.hex()}")
7
8                 if lastchar==b'\xfe': #EOL
9                     print("\n Pacchetto completo ricevuto da Arduino")
10                    self.useData()
11                    self.inbuffer =[]
12                else:
13
14                    self.inbuffer.append (lastchar)
15            else:
16                if self.ser is None:
17                    print("Arduino non connesso - nessun dato in arrivo")
18                    time.sleep(5)
19
20 def useData(self):
21     if len(self.inbuffer)<3: # at least header, size, footer
22         return False
23     # split parts
24     if self.inbuffer[0] != b'\xff':
25         return False

```

```

26     numval = int.from_bytes(self.inbuffer[1], byteorder='little')
27
28
29     # Message structure: FF + num_sensori + (tipo_sensore + valore) *
numval + FE
30     for i in range(numval):
31         sensor_offset = 2 + i * 2
32
33         if len(self.inbuffer) >= sensor_offset + 2:
34             sensor_type_char = self.inbuffer[sensor_offset].decode('utf-8')
35             sensor_value = int.from_bytes(self.inbuffer[sensor_offset + 1],
byteorder='little')
36
37             sensor_name = SENSOR_MAPPING.get(sensor_type_char, f"sensor_{
sensor_type_char}")
38             strval = "Sensor %s (%s): %d " % (sensor_name, sensor_type_char
, sensor_value)
39             print(strval)
40
41             topic = MqttConfigurationParameters.build_telemetry_plant_topic
(sensor_name)
42             payload = '{:d}'.format(sensor_value)
43             print(f"MQTT publish - Topic: {topic}, Payload: {payload}")
44             self.clientMQTT.publish(topic, payload)

```

Per analizzare il flusso di comunicazione tra il microcontrollore, bridge e il broker si rimanda al seguente sequence diagram.

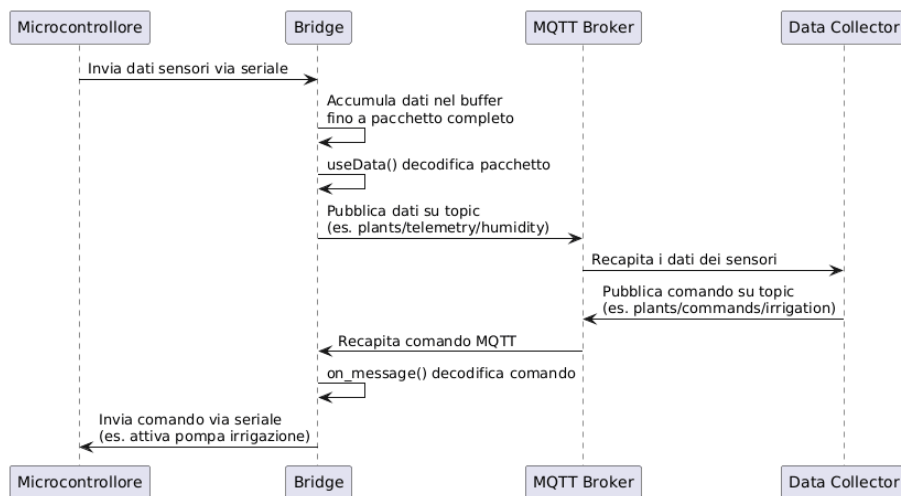


Figura 3.6: Sequence diagram relativo alla classe bridge

3.2.1 Perché si è scelto di utilizzare il protocollo MQTT?

Per far comunicare il bridge (in locale) e il data collector (in cloud), abbiamo optato per il protocollo MQTT (Message Queuing Telemetry Transport), uno degli standard più utilizzati nel mondo dell'Internet of Things [2]. Questa scelta è stata motivata dalle diverse caratteristiche che lo rendono particolarmente adatto a situazioni in cui i dispositivi hanno risorse limitate, come nel caso di Arduino.

In primo luogo, MQTT è un protocollo leggero ed efficiente, progettato appositamente per l'IoT. L'overhead è davvero minimo (l'header di un messaggio occupa solo due byte), il che aiuta a ridurre sia l'uso della banda sia il consumo energetico: due aspetti cruciali per i sistemi wireless o alimentati a batteria.

Dal punto di vista architetturale, MQTT si basa sul modello publish/subscribe. In questo schema, i sensori (publisher) inviano i dati raccolti a un broker, mentre i sistemi che devono elaborare questi dati (subscriber) li ricevono solo se sono interessati ai topic specifici. Ad esempio, in questo progetto, Arduino pubblica i dati di umidità, mentre il sistema centrale li riceve e li elabora. Il broker, in questo caso Mosquitto, agisce da mediatore, assicurando che la comunicazione avvenga in modo asincrono e senza la necessità di collegamenti diretti tra i dispositivi.

Un altro grande vantaggio è la scalabilità. Molti dispositivi possono connettersi al broker contemporaneamente senza necessitare di modifiche significative all'architettura, ed è facile aggiungere nuovi sensori o attuatori semplicemente definendo topic dedicati. Inoltre, la gerarchia dei topic e l'uso di wildcard rendono l'organizzazione dei messaggi flessibile e intuitiva.

Dal punto di vista dell'affidabilità, MQTT offre vari livelli di Quality of Service (QoS) che assicurano la consegna dei messaggi in base alle esigenze specifiche delle applicazioni, ad esempio, in questo prototipo, si è scelto di utilizzare un QoS pari a zero [2].

3.3 Broker MQTT

Il protocollo MQTT offre strumenti utili per gestire e mantenere la comunicazione tra il bridge (in locale) e il data collector (in cloud). Il nucleo della comunicazione è strutturato attorno a un broker MQTT, che riceve i messaggi inviati dai client publisher (sensori e attuatori) e li inoltra ai rispettivi subscriber (dispositivi o servizi) [2].

Per questo progetto, è stato realizzato il broker MQTT utilizzando Mosquitto, un'implementazione open source del protocollo MQTT, conosciuta per la sua leggerezza, stabilità e facilità di integrazione nei sistemi IoT [3]. Mosquitto è stato eseguito all'interno di un contenitore Docker, una piattaforma di containerizzazione che consente di eseguire applicazioni in ambienti isolati e portabili, semplificando così la distribuzione e la gestione dei servizi [4].

La configurazione del broker è gestita tramite docker-compose, nel quale è stato definito un servizio mqtt-broker basato sull'immagine ufficiale eclipse-mosquitto:2.0. Il broker Mosquitto, in esecuzione all'interno del container, è in ascolto sulla porta 1883, per la comunicazione MQTT, ed espone all'host tale servizio sulla porta 7883. Inoltre, è stato abilitato il meccanismo di persistenza, che consente al broker di mantenere informazioni sui client e sugli argomenti anche dopo un riavvio, migliorando così l'affidabilità del sistema. Di seguito viene mostrato il codice del docker-compose:

```
1 services:
2   mqtt-broker:
3     image: eclipse-mosquitto:2.0
4     container_name: mqtt-broker
5     ports:
6       - "7883:1883"    # MQTT TCP
7     volumes:
8       - ./mosquitto/config/mosquitto.conf:/mosquitto/config/mosquitto.conf
9       - ./mosquitto/log:/mosquitto/log
10    restart: unless-stopped
```

Nel file di configurazione `mosquitto.conf` si trovano le seguenti importanti impostazioni:

- `persistence true` e `persistence_location /mosquitto/data`: per gestire la persistenza dei dati;
- `log_dest file /mosquitto/log/mosquitto.log`: per indicare dove si trova il file di log;
- `listener 1883` con `allow_anonymous true`: per permettere accessi anonimi sulla porta standard MQTT.

Di seguito riportiamo le configurazioni del file `mosquitto.conf`:

```
1 persistence true
2 persistence_location /mosquitto/data
3 log_dest file /mosquitto/log/mosquitto.log
4
5 listener 1883
6 allow_anonymous true
```

Per avviare il broker MQTT è necessario assicurarsi che Docker sia in esecuzione sul PC e successivamente eseguire il comando:

```
1 docker-compose up -d
```

Questo comando avvia il contenitore `mqtt-broker` in background, rendendo operativo il broker secondo la configurazione definita nel file `docker-compose.yml` e nel `mosquitto.conf`.

3.4 Plant Descriptor

La classe `PlantDescriptor` rappresenta il modello dati centrale per ogni pianta all'interno del sistema, essa raccoglie tutte le informazioni necessarie per identificare la pianta e per gestire i dispositivi associati: sensori e attuatori.

Gli attributi principali della classe sono:

- **plant_id**: un identificatore univoco per la pianta, utilizzato per collegare i dati dei sensori e degli attuatori alle rispettive entità.
- **species**: la specie della pianta, utile sia per la visualizzazione nel frontend sia per eventuali regolazioni specifiche delle policy.
- **sensors**: una lista di sensori associati alla pianta. Nel prototipo, viene inizializzato almeno un **HumiditySensor**, con parametri quali: valore iniziale, unità di misura, valori minimi e massimi e configurazione realistica o simulata.
- **actuators**: una lista di attuatori collegati alla pianta. Ad esempio, l'attuatore **IrrigationActuator** consente di controllare manualmente o automaticamente l'irrigazione.

La classe fornisce anche un metodo `to_json()`, che converte le informazioni della pianta in formato JSON, rendendo semplice la memorizzazione e lo scambio dei dati tra backend, frontend e data collector. Questo metodo estrae i tipi di sensori e attuatori associati e li include nella rappresentazione JSON insieme a **plant_id** e **species**.

In sintesi, **PlantDescriptor** funge da struttura dati standardizzata per ogni pianta, consentendo a tutte le componenti del sistema di interagire in maniera coerente, modulare e facilmente estensibile.

3.5 Data Collector

Il Data Collector, sviluppato in Python e operante in un ambiente cloud, è progettato per gestire l'invio e la ricezione di messaggi all'interno del sistema. La sua architettura è modulare, il che consente di mantenere separate le varie logiche operative. L'unità data collector è composta da:

- **Consumer**: si occupa della ricezione dei messaggi MQTT dal bridge.
- **Producer**: si occupa di inviare messaggi agli attuatori.
- **Factory**: crea la rappresentazione delle piante utilizzando il Plant Descriptor, partendo da file in formato JSON.
- **Json Manager**: gestisce le operazioni di lettura e scrittura dei file JSON.
- **Policy Manager**: si occupa di valutare automaticamente le policy per le piante, nel caso le policy non siano rispettate vengono generate azioni (trasmesse agli attuatori).

Le diverse logiche sono state organizzate in classi distinte per garantire ordine, chiarezza e una migliore manutenibilità del codice. Nelle prossime sezioni analizzeremo le principali classi e i loro metodi.

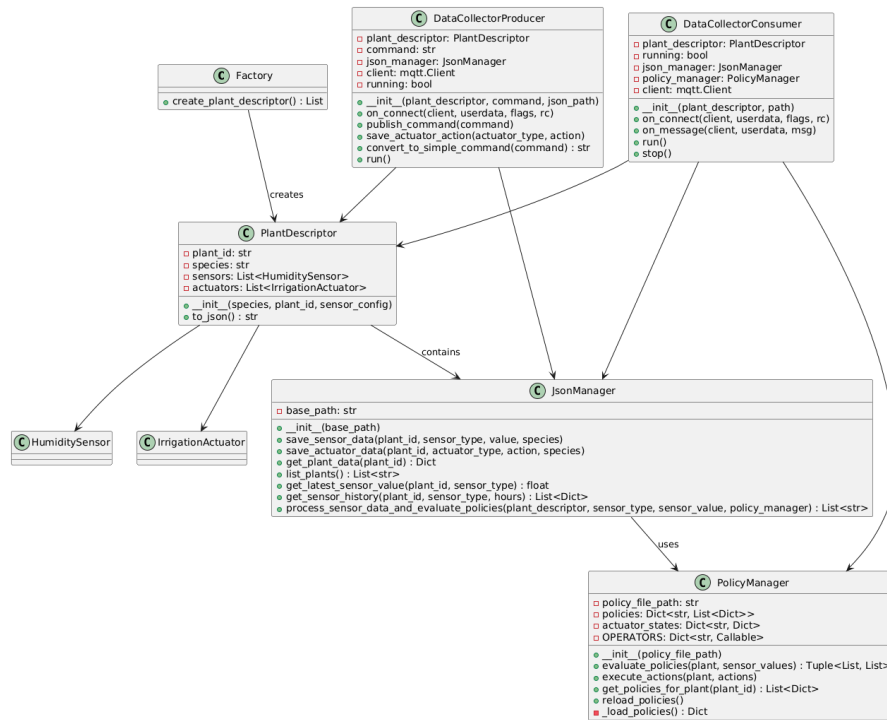


Figura 3.7: Class diagram che contiene le relazioni tra le classi Consumer, Producer, Factory, Json Manager e Policy Manager

3.5.1 Main

Questa è la classe principale che si occupa di orchestrare l'intero sistema e deve essere eseguita per far partire l'intera unità Data collector. In particolare l'istanza Main avvia tutti i consumer e mantiene l'applicazione in esecuzione fino a quando non viene ricevuto un segnale d'interruzione **CTRL+C**, questa classe è costituita da:

- **__init__(config_path)**: crea i Plant Descriptor utilizzando la Factory.
- **start()**: avvia un consumer dedicato per ogni pianta, eseguito in thread separati.
- **stop()**: interrompe i consumer e chiude i thread avviati.

3.5.2 Consumer

Questo componente è responsabile della ricezione dei dati MQTT dai sensori (dialoga con il bridge MQTT) e i suoi principali metodi sono:

- **__init__(plant_descriptor, path)**: inizializza il consumer MQTT.
- **on_connect()**: gestisce la sottoscrizione ai topic dei sensori.

- `on_message()`: elabora i messaggi MQTT ricevuti, convertendo i valori dei sensori, salvandoli tramite il `JsonManager` e valutando eventuali azioni tramite il `PolicyManager`.
- `run()`: avvia il consumer, mantenendo la connessione MQTT attiva e creando un event loop asincrono dedicato alla gestione delle operazioni per la specifica pianta (`plant_id`).
- `stop()`: interrompe il consumer e chiude la connessione MQTT.

Il metodo `run_async_loop()`, definito all'interno di `run()`, ha la funzione di creare e mantenere un event loop asincrono che gestisce le operazioni relative alla pianta in modo continuo ed efficiente, separando l'elaborazione dei messaggi dal flusso principale del programma. Questo permette di eseguire operazioni di lunga durata, come la gestione dei messaggi MQTT e l'interazione con dispositivi esterni, senza bloccare il programma principale.

3.5.3 Producer

Questa classe gestisce la pubblicazione di comandi MQTT degli attuatori delle piante, permettendo al sistema di controllare dispositivi come l'irrigazione automatica basandosi sulle policy valutate. I suoi metodi principali sono:

- `__init__(plant_descriptor, command, json_path)`: inizializza il producer MQTT.
- `on_connect()`: callback per la connessione MQTT.
- `publish_command(command)`: pubblica un comando MQTT.
- `save_actuator_action()`: registra le azioni degli attuatori in formato JSON.
- `convert_to_simple_command()`: converte comandi complessi in comandi semplici e standard.
- `run()`: avvia il producer e pubblica i comandi, invia i comandi per gli attuatori.

3.5.4 Factory

Questa classe gestisce la persistenza dei dati in formato JSON con le seguenti funzioni:

- `save_sensor_data()`: salva i dati dei sensori.
- `save_actuator_data()`: registra le azioni degli attuatori.
- `get_plant_data()`: recupera i dati relativi a una pianta.

- `get_latest_sensor_value()`: restituisce l'ultimo valore registrato da un sensore.
- `get_sensor_history()`: fornisce la cronologia dei dati sensoriali.
- `list_plants()`: elenca tutte le piante monitorate.
- `process_sensor_data_and_evaluate_policies()`: elabora i dati dei sensori e valuta le policy configurate.

3.5.5 Policy Manager

Questo modulo è dedicato alla valutazione delle policy e alla conseguente attivazione di azioni automatiche (attuatori) e all'invio di alert, i suoi metodi principali sono:

- `__init__(policy_file_path)`: carica le policy definite in un file JSON e inizializza lo stato degli attuatori.
- `evaluate_policies(plant, sensor_values)`: valuta le policy per una pianta specifica confrontando i valori dei sensori con le condizioni definite. Restituisce due liste: azioni da eseguire e alert da notificare.
- `execute_actions(plant, actions)`: esegue le azioni generate da `evaluate_policies()`, attivando o disattivando gli attuatori tramite il `DataCollector Producer`.
- `get_policies_for_plant(plant_id)`: restituisce le policy associate a una specifica pianta.
- `reload_policies()`: ricarica le policy dal file JSON.

3.6 File JSON

Per consentire al sistema di leggere e valutare automaticamente le politiche associate a ciascuna pianta monitorata, è stato introdotto un componente chiamato Policy Manager. Le politiche sono definite all'interno di un file in formato JSON, nel quale vengono specificate le condizioni da controllare e le azioni da intraprendere. Nel prototipo, le politiche sono memorizzate all'interno del file `policy_conf.json`:

```
1 [
2   {
3     "plant_id": "my_felce",
4     "policies": [
5       {
6         "sensor": "humidity",
7         "condition": "<",
8         "value": 75,
9         "actuator": "irrigation",
10        "action": "activate"
```

```
11     },
12     {
13         "sensor": "humidity",
14         "condition": ">",
15         "value": 90,
16         "actuator": "irrigation",
17         "action": "deactivate"
18     }
19 ]
20 }
21 ]
```

All'avvio del sistema viene caricato il file `plant_config.json`, da cui vengono inizializzate le variabili e i parametri necessari per il corretto funzionamento del sistema, quello utilizzato dal prototipo ha la seguente struttura:

```
1 {
2     "plants": [
3         {
4             "plant_id": "my_felce",
5             "species": "Pteridofite",
6             "description": "Pianta Felce",
7             "sensors": [ {
8                 "humidity": {
9                     "enabled": true,
10                    "initial_value": 80.0,
11                    "unit": "%",
12                    "min_value": 50.0,
13                    "max_value": 150.0,
14                    "is_real": true
15                }
16            } ],
17            "actuators": [ {
18                "irrigation": {
19                    "enabled": true,
20                    "is_real": true
21                }
22            } ]
23        }
24    ]
25 }
26 }
```

Tutti i dati raccolti dai sensori e lo stato attuale degli attuatori vengono salvati in file JSON dedicati. Per ogni pianta monitorata è previsto un file specifico, se non esiste già, il sistema lo crea automaticamente.

Relativamente al prototipo in cloud viene memorizzato il seguente file `<nome pianta>.json`:

```
1 {
2     "plant_id": "my_felce",
3     "species": "Pteridofite",
4     "created_at": "2025-09-21T16:43:40.677373Z",
5     "last_updated": "2025-09-21T17:39:48.781252Z",
6     "sensors": {
```

```
7   "humidity": [
8     {
9       "value": 73.0,
10      "timestamp": 1758465820
11    },
12    {
13      "value": 73.0,
14      "timestamp": 1758465823
15    },
16    {
17      "value": 80.0,
18      "timestamp": 1758465825
19    },
20    {
21      "value": 81.0,
22      "timestamp": 1758465827
23    }
24  ],
25 },
26 "actuators": {
27   "irrigation": [
28     {
29       "action": "start",
30       "timestamp": 1758465852
31     },
32     {
33       "action": "stop",
34       "timestamp": 1758465855
35     },
36     {
37       "action": "start",
38       "timestamp": 1758466053
39     },
40     {
41       "action": "stop",
42       "timestamp": 1758466062
43     }
44   ]
45 }
46 }
```

3.6.1 Json Manager

Per facilitare la lettura e la scrittura dei file JSON si è deciso di creare una classe chiamata **JsonManager**, che si occupa di gestire completamente i dati. Questa classe offre una serie di metodi che interessano ogni fase del ciclo di vita delle informazioni, ed in particolare:

- La creazione e gestione dei file: alla prima esecuzione, se la directory o i file non sono presenti, vengono creati automaticamente grazie ai metodi `_ensure_directory_exists()` e `_create_empty_plant_data()`.
- Il caricamento dei dati: il metodo `_load_plant_data()` consente di recuperare in modo sicuro i dati esistenti, gestendo eventuali errori di lettura o file danneggiati.

- Il salvataggio: i metodi `save_sensor_data()` e `save_actuator_data()` registrano rispettivamente i valori dai sensori e le azioni degli attuatori, sempre accompagnati da timestamp. Per evitare che i file crescano all'infinito, è possibile impostare un numero massimo di registrazioni da mantenere.
- La consultazione: la classe offre metodi per ottenere i dati di una pianta specifica (`get_plant_data()`), la lista delle piante registrate (`list_plants()`), l'ultimo valore rilevato da un sensore (`get_latest_sensor_value()`) e lo storico dei valori filtrato per intervallo di tempo (`get_sensor_history()`).
- L'integrazione con le politiche di controllo: il metodo `process_sensor_data_and_evaluate_policies()` non solo salva i nuovi dati, ma li utilizza anche per attivare il modulo di Policy Manager, valutando automaticamente eventuali regole predefinite e generando le azioni necessarie.

3.6.2 Perchè si è scelto di utilizzare il formato JSON?

La scelta di adottare il formato JSON (JavaScript Object Notation) per la memorizzazione dei dati nel prototipo è supportata dalle seguenti motivazioni:

- **Semplicità e leggibilità:** JSON presenta una sintassi piuttosto semplice, chiavi e valori, oggetti, array, pertanto è facilmente leggibile e scrivibile, sia per gli esseri umani che per le macchine, rendendo più semplice il debug, la manutenzione e lo sviluppo rapido [7].
- **Efficienza in termini di spazio e di parsing:** Rispetto a formati come l'XML, JSON è generalmente più compatto (con meno "overhead" di tag), e ciò contribuisce a ridurre le dimensioni dei file da salvare e i costi associati allo storage o alla trasmissione; inoltre, il parsing è di solito più veloce, aspetto cruciale per i sistemi che devono continuamente leggere e scrivere dati da sensori o attuatori [8].
- **Compatibilità:** JSON è diventato il formato di riferimento per lo scambio di dati nelle Web API (REST), nei servizi cloud e negli ambienti IoT, pertanto è, una scelta ovvia quando si tratta di esporre dati tramite HTTP, sia per frontend web che per applicazioni mobili, o per interagire con protocolli che utilizzano payload JSON [9].
- **Flessibilità:** JSON consente di creare strutture annidate (oggetti e array), ed è perfetto per rappresentare dati complessi, come quelli provenienti da più sensori, storici, attuatori con azioni e timestamp, infine è ideale per modelli in cui ogni "unità" (come una pianta) ha diversi sensori, attuatori e proprietà accessorie [10].
- **Portabilità:** Essendo un formato standard aperto, JSON può essere letto e scritto in quasi tutti i linguaggi di programmazione moderni, questo facilita

l'interoperabilità tra moduli, componenti cloud, server, frontend, microservizi e così via. JSON è davvero indipendente dalla tecnologia sottostante [11].

3.6.3 Digital Twin e memorizzazione in Cloud

Il concetto di digital twin è la rappresentazione digitale del sistema fisico e consente di progettare la struttura, le funzionalità e lo stato in tempo reale. Nel caso del *vaso intelligente*, il digital twin è implementato tramite file JSON, che descrive in maniera strutturata i dispositivi del sistema (sensori di umidità, relè, pompa, attuatori), le loro proprietà e le logiche operative.

Questo approccio permette di mantenere nel cloud un modello persistente e condiviso del vaso reale, accessibile da diversi moduli software. Nel prototipo, la classe JSON Manager si occupa della lettura e scrittura dei file JSON, utilizzati per aggiornare le informazioni del sistema. I dati memorizzati vengono poi letti e visualizzati nel frontend, tramite un'interfaccia web dedicata agli utenti.

3.7 Backend

Per permettere l'interazione tra il frontend dell'applicazione web e il prototipo del *vaso intelligente*, è stato creato un server REST API utilizzando il framework Flask in Python. Questo server ha il compito di leggere i dati generati dal data collector, che include sensori e attuatori, memorizzati in formato JSON, inoltre espone questi dati attraverso endpoint HTTP standardizzati, rendendo le informazioni facilmente accessibili per le applicazioni web.

Il nucleo logico del sistema è rappresentato dalla classe PlantDataService, la quale, attraverso il modulo JsonManager, gestisce le seguenti operazioni:

- caricamento dei dati delle piante dai file JSON;
- elaborazione dei dati per la presentazione al frontend, includendo informazioni quali:
 - livello di umidità del suolo già normalizzato in percentuale,
 - stato corrente del sistema di irrigazione,
 - timestamp dell'ultima irrigazione,
 - cronologia dei valori provenienti da sensori e attuatori (limitata agli ultimi dieci campioni per ciascun dispositivo).

Il server offre una serie di endpoint REST, sotto elencate, che permettono di interagire con il sistema in modo semplice e diretto:

- GET /api/plants: restituisce un elenco di tutte le piante con il loro stato attuale;
- GET /api/plants/<id>: fornisce i dettagli di una pianta specifica;

- `POST /api/plants/<id>/actuators/<tipo>`: consente di inviare comandi agli attuatori (come accendere o spegnere la pompa di irrigazione) utilizzando il protocollo MQTT, registrando anche l'azione nei file JSON;
- `GET /api/plants/<id>/sensors/<tipo>/history`: restituisce la cronologia dei valori di un sensore specifico su un intervallo di tempo definito;
- `GET /api/health`: fornisce informazioni sullo stato attuale del server;
- `GET /api/stats`: calcola statistiche aggregate, come il numero totale di piante monitorate, le irrigazioni attive e l'umidità media del suolo.

Il server deve gestire inoltre la validazione dei comandi, la corretta formattazione dei payload JSON e si occupa della gestione delle eccezioni, garantendo così robustezza, coerenza e affidabilità nella comunicazione tra il backend e l'interfaccia utente. Grazie a questa architettura modulare e scalabile, il sistema può essere facilmente integrato in applicazioni IoT più ampie, permettendo un monitoraggio in tempo reale delle piante e una gestione automatizzata delle azioni sugli attuatori.

3.7.1 Perché si è scelto di utilizzare il framework Flask?

Per realizzare il server REST API si è scelto Flask, un micro-framework open source per Python, esso può essere definito “micro-framework” in quanto implementa solo le funzionalità essenziali per la creazione di applicazioni web (routing, gestione delle richieste HTTP, risposta JSON, integrazione semplice con moduli esterni) senza imporre un'architettura rigida. Questa leggerezza lo rende particolarmente adatto per applicazioni IoT con requisiti di rapidità di sviluppo e modularità.

Studi recenti hanno mostrato come Flask venga impiegato con successo in contesti di monitoraggio ambientale e IoT, grazie anche alla sua flessibilità e al basso overhead.

Un'analisi comparativa con altri framework, come Django, evidenzia che Flask richiede meno vincoli strutturali, consentendo un maggiore controllo (fine-grained) da parte dello sviluppatore, pur mantenendo una buona capacità di scalare in applicazioni di piccola-media dimensione. [5].

Nell'ambito agricolo e IoT, alcuni studi mostrano come un'applicazione real-time basata su Flask consenta la visualizzazione immediata dei dati dei sensori, semplificando la componente di backend [6].

3.8 Frontend

L'applicazione web è realizzata con Next.js 14 e React, che funge da dashboard per il monitoraggio e per il controllo dell'intero sistema IoT dedicato alle piante. L'interfaccia utente è stata progettata per essere semplice e immediata, permettendo di visualizzare i dati provenienti dai sensori (archiviati in file JSON) e di controllare gli attuatori in tempo reale.

La dashboard è suddivisa in tre sezioni principali:

- La pagina iniziale fornisce una panoramica dello stato delle piante: ogni pianta è rappresentata da una scheda responsive che mostra i dati dei sensori, come umidità, temperatura e luminosità, oltre allo stato dell'attuatore di irrigazione e ai grafici storici dei dati raccolti.
- La visualizzazione dettagliata consente di esplorare i valori sensoriali attraverso grafici interattivi, in particolare grafici a linee per monitorare l'andamento dell'umidità, arricchiti da indicatori visivi come barre di progresso e badge di stato. I dati si aggiornano automaticamente, grazie ad un meccanismo di polling ogni dieci secondi, assicurando così che le informazioni siano sempre aggiornate. L'interfaccia è completamente responsive, adattandosi perfettamente a desktop, tablet e dispositivi mobili.
- La sezione di controllo manuale permette all'utente di attivare o disattivare l'irrigazione tramite pulsante play/stop. Ogni azione è accompagnata da un feedback visivo per rendere l'esperienza utente chiara e semplice.

Il tempo di aggiornamento dell'applicazione è stato impostato su un valore molto veloce per scopi dimostrativi. Durante la fase di produzione, questo intervallo può essere esteso fino a un minuto, sia per raccogliere ed elaborare i dati provenienti da sensori e attuatori, sia per aggiornare le informazioni visualizzate nel frontend.

I componenti principali del frontend sono i seguenti:

- `app/page.tsx`: funge da punto di ingresso per l'applicazione, occupandosi di caricare i dati e gestire lo stato globale. Tra le sue funzioni principali ci sono il recupero delle informazioni sulle piante dal backend e la gestione dei comandi manuali per l'irrigazione e l'aggiornamento regolare delle informazioni tramite polling. Il seguente snippet mostra l'uso di `useEffect`, un hook di React che permette di eseguire effetti collaterali (come il caricamento dei dati e l'avvio di un polling) dopo il rendering del componente:

```
1      useEffect(() => {
2          loadPlantsData()
3
4          // Poll for updates every 10 seconds
5          const interval = setInterval(() => {
6              loadPlantsData()
7          }, 10000)
8
9          return () => clearInterval(interval)
10     }, [])
11
```

- `components/plant-dashboard.tsx`: si occupa di mostrare le card, ognuna delle quali presenta i dati sensoriali, lo stato dell'attuatore e un grafico interattivo. Tra i componenti interni, troviamo:

- PlantCard, che riassume le informazioni di una singola pianta (visualizzazione nome, specie e valore di umidità rilevato) e include i controlli manuali;
 - PlantImage, che gestisce in modo flessibile il caricamento delle immagini permettendo di visualizzare, in base alla specie, l'immagine della pianta specifica; se non è presente l'immagine corrispondente a quella specie ne viene caricata una di default;
 - HumidityChart, che visualizza in tempo reale i dati di umidità attraverso un grafico lineare, realizzato utilizzando la libreria **recharts**. Questo componente supporta aggiornamenti dinamici e interattività.
- Il modulo API service (`lib/api.ts`) si occupa di gestire in modo centralizzato le chiamate HTTP verso il backend ed offre metodi per recuperare i dati delle piante, controllare gli attuatori e consultare la cronologia dei sensori. È realizzato come singleton in TypeScript e include meccanismi per la gestione degli errori e la configurazione tramite variabili d'ambiente.
 - La cartella `components/ui/` contiene i componenti di interfaccia riutilizzabili, come card, pulsanti, barre di progresso e badge. Questi componenti sono stati sviluppati utilizzando Radix UI e ottimizzati con Tailwind CSS, per garantire una coerenza visiva e una buona riusabilità del codice.

3.8.1 Tecnologie utilizzate

Lo sviluppo del frontend del sistema è stato realizzato utilizzando alcune delle tecnologie più diffuse e consolidate per la creazione di interfacce web moderne, tra cui:

- **TypeScript** è un linguaggio di programmazione che estende JavaScript, introducendo un sistema di tipizzazione statica che consente di individuare errori già in fase di compilazione e migliora la manutenibilità del codice, rendendo lo sviluppo più robusto e scalabile [12].
- **React** è una libreria open source sviluppata da Meta, utilizzata per costruire interfacce utente basate su componenti riutilizzabili. La sua architettura dichiarativa semplifica la gestione dello stato e il rendering dinamico degli elementi, rendendola una delle soluzioni più popolari nello sviluppo web [13].
- **Next.js** è un framework che estende React, offrendo funzionalità avanzate come il rendering lato server (server-side rendering), il supporto nativo al routing e l'ottimizzazione delle performance. Queste caratteristiche lo rendono particolarmente adatto per realizzare applicazioni scalabili e veloci, come dashboard interattive per sistemi IoT [14].

- Per visualizzare i dati sensoriali, è stata adottata la libreria **Recharts**, costruita su React e progettata per creare grafici interattivi. Essa permette di rappresentare i dati storici in tempo reale, in modo chiaro e leggibile, garantendo anche la possibilità di interazione da parte dell'utente [15].

L'adozione di queste tecnologie ha reso possibile la creazione di un'interfaccia moderna, responsive e facilmente estendibile, capace di integrare dati eterogenei provenienti dal sistema IoT e di renderli accessibili in tempo reale.

Capitolo 4

Sviluppi Futuri

4.1 Integrazione di sensori aggiuntivi

L'attuale prototipo è una versione base, focalizzata principalmente sul monitoraggio dell'umidità del terreno. Un'ulteriore sviluppo del prototipo potrebbe prevedere l'integrazione di altri sensori, ad esempio quelli per la luminosità (per valutare l'esposizione solare della pianta), la temperatura ambientale e il pH del terreno, in modo da avere un controllo ad ampio spettro di tutte le condizioni che incidono sulla crescita e sul benessere della pianta.

4.2 Notifiche push

Il prototipo potrebbe essere potenziato con un sistema di gestione utenti, completo di autenticazione e login dedicati; questo consentirebbe a ciascun utente di monitorare solo le proprie piante, ricevendo notifiche push personalizzate direttamente sullo smartphone o sul tablet. Le notifiche potrebbero segnalare eventuali situazioni critiche, come un'eccessiva o insufficiente esposizione al sole, sbalzi di temperatura anomali o la necessità di riempire il serbatoio dell'acqua. Questo approccio renderebbe il sistema molto più fruibile, avvicinandolo alle reali esigenze degli utenti finali e flessibile alle diverse esigenze degli utenti.

4.3 Miglioramento delle animazioni e integrazione multi-vaso

Il prototipo potrebbe essere migliorato potenziando la parte visiva del sistema, sfruttando la matrice LED già presente, oppure esplorando schede di sviluppo alternative con capacità grafiche superiori; si potrebbero introdurre nuove animazioni che comunicano in modo immediato lo stato della pianta. Ad esempio, rappresentazioni visive potrebbero mostrare non solo quando l'irrigazione è attivata, ma anche parametri come temperatura, luminosità o livello del serbatoio.

Inoltre, il progetto potrebbe essere esteso alla gestione multi-vaso di piante che utilizzano lo stesso sistema di monitoraggio ambientale (come un unico sensore di luminosità e temperatura per vasi vicini) o un serbatoio centralizzato per l'irrigazione. Questo approccio ridurrebbe i costi complessivi e renderebbe il sistema più efficiente e scalabile.

4.4 Una migliore gestione delle policy

Un altro possibile sviluppo potrebbe semplificare la gestione delle regole di irrigazione (policy), che attualmente richiedono la modifica manuale di file di configurazione in formato JSON. Con un'interfaccia grafica dedicata, gli utenti potrebbero definire o modificare le policy direttamente dall'applicazione web, senza dover intervenire su file di testo o avere conoscenze tecniche specifiche.

Inoltre, il sistema potrebbe includere un database di varie specie vegetali, permettendo di selezionare la pianta presente nel vaso. In questo modo, il programma caricherebbe automaticamente le policy di irrigazione e i parametri ottimali di umidità, temperatura e luminosità per quella specifica pianta. Questa funzionalità renderebbe il sistema più flessibile, intuitivo e accessibile a un pubblico più ampio, migliorando l'esperienza d'uso e aumentando l'efficacia nella gestione personalizzata delle piante.

4.5 Algoritmi predittivi per l'irrigazione basati su machine learning

I dati raccolti potrebbero essere archiviati non solo in semplici file JSON, ma anche in database professionali come PostgreSQL o InfluxDB, permettendo così analisi più avanzate, scalabilità e una gestione delle informazioni più affidabile.

L'estrazione di dataset, a partire da database, potrebbe aprire la strada all'uso di tecniche di intelligenza artificiale e machine learning, rendendo il sistema davvero "smart". In particolare, gli algoritmi predittivi sarebbero in grado di comprendere le esigenze idriche delle diverse specie vegetali, tenendo conto di fattori come l'esposizione al sole, la temperatura e l'umidità del suolo.

In questo modo, il vaso intelligente potrebbe gestire l'irrigazione in modo autonomo, ottimizzando l'uso dell'acqua e migliorando la salute generale delle piante. Inoltre, un'analisi storica dei dati consentirebbe di sviluppare modelli adattivi e personalizzati per ogni utente o contesto ambientale.

Capitolo 5

Conclusioni

Il progetto realizzato può essere considerato un primo prototipo parte di un sistema intelligente per gestire l'irrigazione delle piante. Questo sistema è in grado di integrare sensori, attuatori e protocolli di comunicazione tipici dell'Internet of Things comprensivo dello sviluppo hardware.

5.1 Limiti

L'applicazione presenta alcune limitazioni che attualmente ne ostacolano la distribuzione su larga scala. In particolare, mancano le funzionalità essenziali come la sicurezza dei dati, la gestione degli utenti e un sistema di automazione delle policy che elimini la necessità di interagire manualmente con file di configurazione in formato JSON. Inoltre, il sistema non segue standard consolidati per la rappresentazione dei dati, come SenML, ed è richiesto il collegamento fisico tra la scheda Arduino e il bridge via cavo, ciò rappresenta un limite per la flessibilità complessiva.

5.2 Potenzialità

Il prototipo è molto flessibile, poichè consente di aggiungere facilmente nuovi sensori, attuatori o vasi senza dover stravolgere l'architettura esistente. Inoltre, l'uso del protocollo MQTT, ormai considerato uno standard nel mondo dell'IoT, si adatta perfettamente a questo contesto, grazie alla sua leggerezza e alla sua capacità di gestire in modo efficace le comunicazioni asincrone tra i vari dispositivi.

In parallelo, l'uso del protocollo HTTP per l'interfaccia web segue uno standard ben consolidato, assicurando così la compatibilità con le tecnologie già esistenti e facilitando l'accesso al sistema da qualsiasi browser o dispositivo connesso.

5.3 Considerazioni finali

Il prototipo realizzato è un punto di partenza per ulteriori sviluppi ed estensioni. Creare questo prototipo mi ha dato l'opportunità di mettere in pratica le conoscenze

teoriche che ho acquisito durante il corso di Internet of Things, inoltre mi ha consentito di cimentarmi con sfide tipiche della progettazione hardware e dell'integrazione tra software e hardware.

Implementando un sistema di monitoraggio e irrigazione intelligente, ho potuto dimostrare come le tecnologie IoT possano offrire soluzioni efficaci e innovative, anche in un contesto apparentemente semplice come la cura delle piante.

L'obiettivo principale di questo progetto, sia formativo che sperimentale, è stato raggiunto: il progetto ha dimostrato la fattibilità tecnica e il valore didattico di un sistema IoT di questo tipo, aprendo la strada a potenziali sviluppi futuri, come l'integrazione di nuovi sensori, l'analisi predittiva dei dati e la creazione di un'interfaccia utente sempre più intuitiva e interattiva.

Appendice A

Codice del progetto

In questa appendice sono riportati i principali file che costituiscono il prototipo descritto nel Capitolo 3. Per ciascun componente viene fornito il codice sorgente completo al fine di permettere una comprensione diretta della struttura, delle logiche implementative e delle interazioni tra le diverse parti del sistema. L'ordine dei file rispecchia l'architettura del progetto, partendo dai moduli di comunicazione, passando per la gestione dei dati e delle policy, fino ad arrivare al backend e al frontend dell'applicazione web.

A.1 Bridge

A.1.1 bridge_Serial_MQTT.py

```
1 import serial
2 import serial.tools.list_ports
3 import configparser
4 import sys
5 import os
6 import time
7
8 # Aggiungi il path per importare i moduli del progetto
9 sys.path.append(os.path.dirname(os.path.dirname(os.path.abspath(__file__)))
10 )
11 from conf.mqtt_conf_params import MqttConfigurationParameters
12 import paho.mqtt.client as mqtt
13 SENSOR_MAPPING = {
14     'H': "humidity",
15     'T': "temperature",
16     'L': "lightness",
17     'B': "battery_level"
18 }
19 ACTUATOR_MAPPING = {
20     0: "irrigation", # ID Nome
21 }
22
23 ACTUATOR_CHAR_MAPPING = {
24     0: b'I', # irrigation
25 }
```

```

26 }
27
28 class Bridge():
29
30     def __init__(self):
31         self.config = configparser.ConfigParser()
32         self.config.read('config.ini')
33         self.setupSerial()
34         self.setupMQTT()
35
36     def setupSerial(self):
37         # open serial port
38         self.ser = None
39         self.portname = None
40
41         if self.config.get("Serial", "UseDescription", fallback=False):
42             self.portname = self.config.get("Serial", "PortName", fallback="COM5")
43         else:
44             print("list of available ports: ")
45             ports = serial.tools.list_ports.comports()
46
47             for port in ports:
48                 print (port.device)
49                 print (port.description)
50                 if self.config.get("Serial", "PortDescription", fallback="arduino").
                    lower() \
51                     in port.description.lower():
52                     self.portname = port.device
53
54             # Se non trova la porta per descrizione, usa PortName
55             if self.portname is None:
56                 self.portname = self.config.get("Serial", "PortName", fallback="COM5")
57                 print(f"Usando porta configurata: {self.portname}")
58
59             try:
60                 if self.portname is not None:
61                     print ("connecting to " + self.portname)
62                     self.ser = serial.Serial(self.portname, 9600, timeout=0)
63                     print(f"    Connesso alla porta {self.portname}")
64                 else:
65                     print("    Nessuna porta trovata")
66             except Exception as e:
67                 print(f"    Errore connessione seriale: {e}")
68                 self.ser = None
69
70             # self.ser.open()
71
72             # internal input buffer from serial
73             self.inbuffer = []
74
75     def setupMQTT(self):
76         self.clientMQTT = mqtt.Client()
77         self.clientMQTT.on_connect = self.on_connect
78         self.clientMQTT.on_message = self.on_message
79         print("connecting to MQTT broker...")
80         self.clientMQTT.connect(
81             self.config.get("MQTT", "Server", fallback= "localhost"),
82             self.config.getint("MQTT", "Port", fallback= 7883),
83             60)
84
85         self.clientMQTT.loop_start()
86
87     def on_connect(self, client, userdata, flags, rc):
88         print("Connected with result code " + str(rc))

```

```

89
90 for actuator_id, actuator_name in ACTUATOR_MAPPING.items():
91     topic = MqttConfigurationParameters.build_command_plant_topic(
92         actuator_name)
93     self.clientMQTT.subscribe(topic)
94     print(f"    Sottoscritto a: {topic}")
95
96 def on_message(self, client, userdata, msg):
97     print(f"Comando ricevuto - Topic: {msg.topic}, Payload: {msg.payload}")
98
99     topic_parts = msg.topic.split('/')
100     actuator_type = topic_parts[-1] # "irrigation", "lighting", etc.
101
102     actuator_id = None
103     for id, name in ACTUATOR_MAPPING.items():
104         if name == actuator_type:
105             actuator_id = id
106             break
107
108     if actuator_id is None:
109         print(f"Attuatore non riconosciuto: {actuator_type}")
110         return
111
112     actuator_char = ACTUATOR_CHAR_MAPPING.get(actuator_id)
113     if not actuator_char:
114         print(f"Carattere seriale non trovato per attuatore {actuator_id}")
115         return
116
117     command = msg.payload.decode('utf-8').lower()
118
119     if self.ser is not None:
120         if command == "start" or command == "on" or command == "1":
121             self.ser.write(actuator_char) # Tipo attuatore
122             self.ser.write(b'A') # Comando ATTIVA
123             print(f"Comando ATTIVA inviato per {actuator_type}")
124         elif command == "stop" or command == "off" or command == "0":
125             self.ser.write(actuator_char) # Tipo attuatore
126             self.ser.write(b'S') # Comando DISATTIVA
127             print(f"Comando DISATTIVA inviato per {actuator_type}")
128         else:
129             print(f"Comando non riconosciuto: {command}")
130     else:
131         print("Serial port not available!")
132
133 def loop(self):
134     while (True):
135         if not self.ser is None:
136             if self.ser.in_waiting>0:
137                 lastchar=self.ser.read(1)
138                 print(f"    Byte ricevuto da Arduino: {lastchar.hex()}")
139
140                 if lastchar==b'\xfe': #EOL
141                     print("\n    Pacchetto completo ricevuto da Arduino")
142                     self.useData()
143                     self.inbuffer = []
144                 else:
145                     self.inbuffer.append (lastchar)
146             else:
147                 if self.ser is None:
148                     print("    Arduino non connesso - nessun dato in arrivo")
149                     time.sleep(5)
150
151 def useData(self):

```

```

152 if len(self.inbuffer)<3:    # at least header, size, footer
153     return False
154 # split parts
155 if self.inbuffer[0] != b'\xff':
156     return False
157
158 numval = int.from_bytes(self.inbuffer[1], byteorder='little')
159
160 # Message structure: FF + num_sensori + (tipo_sensore + valore) * numval
161   + FE
162 for i in range(numval):
163     sensor_offset = 2 + i * 2
164
165     if len(self.inbuffer) >= sensor_offset + 2:
166         sensor_type_char = self.inbuffer[sensor_offset].decode('utf-8')
167         sensor_value = int.from_bytes(self.inbuffer[sensor_offset + 1],
168                                     byteorder='little')
169
170         sensor_name = SENSOR_MAPPING.get(sensor_type_char, f"sensor_{
171         sensor_type_char}")
172         strval = "Sensor %s (%s): %d " % (sensor_name, sensor_type_char,
173         sensor_value)
174         print(strval)
175
176         topic = MqttConfigurationParameters.build_telemetry_plant_topic(
177         sensor_name)
178         payload = '{:d}'.format(sensor_value)
179         print(f"MQTT publish - Topic: {topic}, Payload: {payload}")
180         self.clientMQTT.publish(topic, payload)
181
182 if __name__ == '__main__':
183     try:
184         print("Starting Serial-MQTT Bridge...")
185         print("Arduino -> MQTT connection")
186         print("Press Ctrl+C to stop")
187         print("=" * 50)
188
189         br = Bridge()
190         br.loop()
191
192 except KeyboardInterrupt:
193     print("\n Keyboard interrupt (Ctrl+C)")
194     print("Closing connections...")
195
196 # Close serial connection
197 if hasattr(br, 'ser') and br.ser is not None:
198     br.ser.close()
199     print("Serial connection closed")
200
201 # Close MQTT connection
202 if hasattr(br, 'clientMQTT'):
203     br.clientMQTT.loop_stop()
204     br.clientMQTT.disconnect()
205     print("MQTT connection closed")
206
207 print("Bridge terminated successfully")
208
209
210

```

A.2 Plant Descriptor

A.2.1 plant_descriptor.py

```
1 import json
2 from smart_objects.sensors.humidity_sensor import HumiditySensor
3 from smart_objects.actuators.irrigation_actuator import IrrigationActuator
4
5
6 class PlantDescriptor:
7     def __init__(self, species, plant_id, devices=None, sensor_config=None)
8         :
9             self.plant_id = plant_id
10            self.species = species
11
12            if sensor_config:
13                self.sensors = [HumiditySensor(
14                    plant_id,
15                    sensor_config.get("initial_value", 80.0),
16                    sensor_config.get("unit", "%"),
17                    sensor_config.get("min_value", 50.0),
18                    sensor_config.get("max_value", 160.0),
19                    "environment_telemetry",
20                    sensor_config.get("is_real", True)
21                )]
22            else:
23                self.sensors = [HumiditySensor(plant_id, 80.0, "%", 50.0,
24                    160.0, "environment_telemetry", True)]
25
26            self.actuators = [IrrigationActuator(plant_id, "water_metering")]
27
28            def to_json(self):
29
30                all_sensors = []
31                all_actuators = []
32
33                all_sensors.extend([sensor.type for sensor in self.sensors])
34                all_actuators.extend([actuator.type for actuator in self.actuators
35                ])
36
37                return json.dumps({
38                    "plant_id": self.plant_id,
39                    "species": self.species,
40                    "sensors": all_sensors,
41                    "actuators": all_actuators
42                })
```

A.3 Data Collector

A.3.1 data_collector_main.py

```
1 import logging
2 import sys
3 import os
```

```

4 project_root = os.path.abspath(os.path.join(os.path.dirname(__file__), '..')
5 )
6 sys.path.insert(0, project_root)
7 import time
8 import threading
9 from data_collector.factory.factory import Factory
10 from data_collector.data_collector_consumer import DataCollectorConsumer
11
12 PATH="cloud_simulator/plants_log/"
13
14 class Main:
15     """
16     Classe principale per la gestione dei consumer dei dati delle piante..
17     """
18     def __init__(self, config_path):
19         print("    Creazione plant descriptors...")
20         try:
21             self.plants = Factory.create_plant_descriptor()
22             print(f"    Creati {len(self.plants)} plant descriptors")
23             for plant in self.plants:
24                 print(f"    - {plant.plant_id} ({plant.species})")
25         except Exception as e:
26             print(f"    Errore creazione plant descriptors: {e}")
27             import traceback
28             traceback.print_exc()
29             self.plants = []
30
31         self.threads = []
32         self.consumers = []
33
34     def start(self):
35         """
36         Avvia i consumer per ogni pianta.
37         """
38         print(f"    Avvio {len(self.plants)} consumer...")
39         for i, plant in enumerate(self.plants):
40             print(f"    Avvio consumer {i+1}/{len(self.plants)} per {plant
41 .plant_id}")
42             try:
43                 consumer = DataCollectorConsumer(plant, PATH)
44                 t = threading.Thread(target=consumer.run)
45                 t.start()
46                 self.threads.append(t)
47                 self.consumers.append(consumer)
48                 print(f"    Consumer {plant.plant_id} avviato")
49             except Exception as e:
50                 print(f"    Errore avvio consumer {plant.plant_id}: {e}")
51
52         import traceback
53         traceback.print_exc()
54
55     def stop(self):
56         """
57         Interrompe i consumer e attende la terminazione dei thread.
58         """
59         for consumer in self.consumers:
60             if hasattr(consumer, "stop"):
61                 consumer.stop()
62         for t in self.threads:
63             t.join()
64
65 if __name__ == "__main__":
66     """

```

```

65 Entry point principale dell'applicazione data collector.
66 """
67 config_file = "data_collector/factory/plants_config.json"
68 if not os.path.exists(config_file):
69     print(f"    File di configurazione non trovato: {config_file}")
70     sys.exit(1)
71
72 print(f"    File di configurazione trovato: {config_file}")
73
74 print("    Creazione Main...")
75 manager=Main(config_file)
76 print("    Avvio consumer...")
77 manager.start()
78 print("    Sistema in esecuzione...")
79 try:
80     while True:
81         time.sleep(1) # Sleep per evitare CPU al 100%
82 except KeyboardInterrupt:
83     print("\n    Interruzione da tastiera (Ctrl+C)")
84     print("Stopping all threads...")
85     manager.stop()
86 except Exception as e:
87     print(f"\n    Errore durante l'esecuzione: {e}")
88     import traceback
89     traceback.print_exc()
90     print("Stopping all threads...")
91     manager.stop()

```

A.3.2 data_collector_consumer.py

```

1 import logging
2 import time
3 import paho.mqtt.client as mqtt
4 import json
5 import os
6 import asyncio
7 import threading
8 import concurrent.futures
9 from conf.mqtt_conf_params import MqttConfigurationParameters
10 from data_collector.plant_descriptor import PlantDescriptor
11 from data_collector.policy_manager import PolicyManager
12 from data_collector.data_collector_producer import DataCollectorProducer
13 from data_collector.json_manager import JsonManager
14
15 class DataCollectorConsumer:
16     """
17     Consumer MQTT per la raccolta e elaborazione dei dati delle piante.
18     """
19
20     def __init__(self, plant_descriptor: PlantDescriptor, path: str):
21
22         self.plant_descriptor = plant_descriptor
23         self.running = False
24
25         self.json_manager = JsonManager(base_path=path)
26
27         self.policy_manager = PolicyManager()
28
29         client_id = f"{plant_descriptor.plant_id}-data-collector-consumer"
30         try:

```

```

31         self.client = mqtt.Client(mqtt.CallbackAPIVersion.VERSION1,
32         client_id)
33         except AttributeError:
34             self.client = mqtt.Client(client_id)
35
36         self.client.on_connect = self.on_connect
37         self.client.on_message = self.on_message
38
39     def on_connect(self, client, userdata, flags, rc):
40         """Callback per la connessione MQTT."""
41         logging.info("Connected with result code %s", str(rc))
42         if rc == 0:
43
44             for sensor in self.plant_descriptor.sensors:
45                 topic = MqttConfigurationParameters.
46                 build_telemetry_plant_topic(
47                     sensor.type
48                 )
49                 self.client.subscribe(topic)
50                 print(f"      Sottoscritto al topic: {topic}")
51                 logging.info(f"Subscribed to topic: {topic}")
52
53     def on_message(self, client, userdata, msg):
54         """
55         Callback per la ricezione di messaggi MQTT.
56
57         Gestisce payload semplici dal bridge (solo valore numerico) e li
58         converte
59         nel formato interno del sistema.
60         """
61         if not self.running:
62             return
63
64         try:
65             message_payload = msg.payload.decode("utf-8")
66             print(f"      Messaggio ricevuto - Topic: {msg.topic}, Payload:
67             {message_payload}")
68             logging.debug(f"Received message on topic {msg.topic}")
69
70             # Topic formato: plant/sensor/{sensor_type}
71             topic_parts = msg.topic.split('/')
72             sensor_type = topic_parts[-1] if len(topic_parts) >= 3 else "
73             unknown"
74
75             try:
76                 sensor_value = float(message_payload)
77             except ValueError:
78                 logging.warning(f"Invalid sensor value: {message_payload}")
79                 return
80
81             if sensor_type == "humidity":
82                 humidity_sensor = None
83                 for sensor in self.plant_descriptor.sensors:
84                     if sensor.type == "humidity":
85                         humidity_sensor = sensor
86                         break
87
88                 if humidity_sensor:
89                     sensor_value = humidity_sensor.
90                     calculate_relative_percentage(sensor_value)
91
92             message_data = {
93                 'topic': msg.topic,

```

```

89         'payload': message_payload,
90         'sensor_type': sensor_type,
91         'value': sensor_value,
92         'device_name': 'environment_telemetry',
93         'timestamp': int(time.time())
94     }
95
96     logging.info(f"Processed sensor data: {sensor_type} = {
sensor_value}")
97
98     actions = self.json_manager.
process_sensor_data_and_evaluate_policies(
99         plant_descriptor=self.plant_descriptor,
100         sensor_type=sensor_type,
101         sensor_value=sensor_value,
102         policy_manager=self.policy_manager
103     )
104
105     for action_str in actions:
106         print(f"      Esecuzione azione per {self.plant_descriptor.
plant_id}: {action_str}")
107
108         if "activate" in action_str.lower() or "deactivate" in
action_str.lower():
109             try:
110                 producer = DataCollectorProducer(
111                     plant_descriptor=self.plant_descriptor,
112                     command=action_str,
113                     json_path=self.json_manager.base_path
114                 )
115                 producer.run()
116                 print(f"      Azione eseguita: {action_str}")
117             except Exception as e:
118                 logging.error(f"Errore esecuzione azione {
action_str}: {e}")
119             else:
120                 print(f"      Azione ignorata (non activate/deactivate
): {action_str}")
121
122         except Exception as e:
123             logging.error(f"Error in on_message: {e}")
124
125     def run(self):
126         """
127         Avvia il consumer MQTT e mantiene la connessione attiva.
128
129         """
130         print(f"      Avvio DataCollectorConsumer per {self.plant_descriptor
.plant_id}")
131
132         try:
133             def run_async_loop():
134                 print(f"      Avvio event loop asyncio per {self.
plant_descriptor.plant_id}")
135                 self.loop = asyncio.new_event_loop()
136                 asyncio.set_event_loop(self.loop)
137                 self.loop.run_forever()
138
139             async_thread = threading.Thread(target=run_async_loop, daemon=
True)
140             async_thread.start()
141
142             time.sleep(0.1)
143

```

```

144         print(f"    Connessione MQTT per {self.plant_descriptor.
plant_id}...")
145         self.client.connect(MqttConfigurationParameters.BROKER_ADDRESS,
MqttConfigurationParameters.BROKER_PORT)
146         self.client.loop_start()
147         self.running = True
148         print(f"    DataCollectorConsumer {self.plant_descriptor.
plant_id} avviato correttamente")
149
150     except Exception as e:
151         print(f"    Errore avvio DataCollectorConsumer {self.
plant_descriptor.plant_id}: {e}")
152         raise
153
154
155     def stop(self):
156         """
157         Interrompe il consumer MQTT e termina la connessione.
158
159         """
160         logging.info("Stopping DataCollectorConsumer...")
161         self.running = False
162
163         try:
164             self.client.disconnect()
165         except Exception as e:
166             logging.warning(f"Error disconnecting MQTT client: {e}")

```

A.3.3 data_collector_producer.py

```

1 import paho.mqtt.client as mqtt
2 from conf.mqtt_conf_params import MqttConfigurationParameters
3 from data_collector.plant_descriptor import PlantDescriptor
4 from data_collector.json_manager import JsonManager
5 import logging
6 import time
7
8 class DataCollectorProducer:
9     """
10     Producer MQTT per l'invio di comandi agli attuatori delle piante.
11
12     """
13     def __init__(self, plant_descriptor: PlantDescriptor, command: str,
json_path: str = "cloud_simulator/plants_log"):
14         self.plant_descriptor = plant_descriptor
15         self.command = command
16
17         self.json_manager = JsonManager(base_path=json_path)
18
19         client_id = f"{self.plant_descriptor.plant_id}-data-collector-
producer"
20         try:
21             self.client = mqtt.Client(mqtt.CallbackAPIVersion.VERSION1,
client_id)
22         except AttributeError:
23             self.client = mqtt.Client(client_id)
24         self.client.on_connect = self.on_connect
25         self.running = True
26
27

```

```

28 def on_connect(self, client, userdata, flags, rc):
29     """
30     Callback per la connessione MQTT.
31
32     """
33     logging.info("Connected with result code %s", str(rc))
34
35 def publish_command(self, command: str):
36     """
37     Pubblica un comando MQTT agli attuatori delle piante.
38
39     """
40     simple_command = self.convert_to_simple_command(command)
41
42     if simple_command:
43         actuator_type = "irrigation" # Default per ora
44
45         topic = MqttConfigurationParameters.build_command_plant_topic(
46             actuator_type)
47         self.client.publish(topic, simple_command)
48         logging.info("Published command: %s to topic: %s",
49             simple_command, topic)
50
51         self.save_actuator_action(actuator_type, simple_command)
52
53 def save_actuator_action(self, actuator_type: str, action: str):
54     """
55     Salva l'azione di un attuttore nel file JSON.
56
57     Args:
58         actuator_type: Tipo di attuttore (es. "irrigation")
59         action: Azione eseguita (es. "start", "stop")
60     """
61     try:
62         self.json_manager.save_actuator_data(
63             plant_id=self.plant_descriptor.plant_id,
64             actuator_type=actuator_type,
65             action=action,
66             species=self.plant_descriptor.species
67         )
68         print(f"        Azione attuttore salvata: {actuator_type} = {
69             action}")
70     except Exception as e:
71         logging.error(f"Errore salvataggio attuttore {actuator_type}: {
72             e}")
73
74 def convert_to_simple_command(self, command: str):
75     """
76     Converte comandi complessi in formato semplice per il bridge.
77
78     Args:
79         command: Comando complesso (es. "Activate irrigation", "
80             Deactivate irrigation")
81
82     Returns:
83         str: Comando semplice (es. "start", "stop")
84     """
85     command_lower = command.lower()
86
87     if "deactivate" in command_lower or "stop" in command_lower or "off
88 " in command_lower:
89         return "stop"
90     elif "activate" in command_lower or "start" in command_lower or "on
91 " in command_lower:

```

```

85         return "start"
86     else:
87         if command_lower in ["start", "stop", "on", "off", "1", "0"]:
88             return command_lower
89         else:
90             logging.warning(f"Unknown command format: {command}")
91             return None
92
93     def run(self):
94         """
95         Avvia il producer MQTT e pubblica il comando.
96
97         """
98         self.client.connect(MqttConfigurationParameters.BROKER_ADDRESS,
99                             MqttConfigurationParameters.BROKER_PORT)
100         self.client.loop_start()
101         self.publish_command(self.command)
102         time.sleep(2)
103         self.client.loop_stop()

```

A.3.4 factory.py

```

1  import json
2  import os
3  from data_collector.plant_descriptor import PlantDescriptor
4  FILEPATH = "plants_config.json"
5  class Factory:
6      """ create the plant descriptor from the json file """
7
8      @staticmethod
9      def create_plant_descriptor():
10         """Legge la configurazione dal file plants_config.json"""
11         config_path = os.path.join(os.path.dirname(__file__), FILEPATH)
12
13         try:
14             with open(config_path, 'r', encoding='utf-8') as f:
15                 config = json.load(f)
16
17                 plant_descriptors = []
18
19                 for plant_config in config.get("plants", []):
20                     plant_id = plant_config.get("plant_id")
21                     species = plant_config.get("species")
22
23                     if plant_id and species:
24
25                         sensor_config = None
26                         sensors_config = plant_config.get("sensors", [])
27                         if sensors_config and len(sensors_config) > 0:
28                             sensor_config = sensors_config[0].get("humidity")
29
30                         plant_descriptor = PlantDescriptor(species, plant_id,
31                                                             sensor_config=sensor_config)
32                         plant_descriptors.append(plant_descriptor)
33                         print(f"    Creato plant descriptor: {plant_id} ({species})")
34
35                         if sensor_config:
36                             print(f"        Sensore umidit : min={sensor_config.get('min_value')}, max={sensor_config.get('max_value')}")
37                         else:

```

```

36         print(f"          Configurazione pianta incompleta: {
plant_config}")
37
38         return plant_descriptors
39
40     except FileNotFoundError:
41         print(f"          File di configurazione non trovato: {config_path}")
42         species = "cactus"
43         plant_id = "plant_cactus_001"
44         plant_descriptor = PlantDescriptor(species, plant_id)
45         return [plant_descriptor]
46
47     except json.JSONDecodeError as e:
48         print(f"          Errore parsing JSON: {e}")
49         species = "cactus"
50         plant_id = "plant_cactus_001"
51         plant_descriptor = PlantDescriptor(species, plant_id)
52         return [plant_descriptor]

```

A.3.5 policy_manager.py

```

1  import json
2  import os
3  import sys
4  import logging
5  from typing import Dict, List, Tuple, Optional
6  project_root = os.path.abspath(os.path.join(os.path.dirname(__file__), '..'
    ))
7  sys.path.insert(0, project_root)
8  from data_collector.plant_descriptor import PlantDescriptor
9  from data_collector.data_collector_producer import DataCollectorProducer
10
11 class PolicyManager:
12     """
13     Gestore semplificato delle policy per la valutazione automatica delle
    condizioni.
14
15     """
16
17     OPERATORS = {
18         "<": lambda x, y: x < y,
19         ">": lambda x, y: x > y,
20         "=": lambda x, y: x == y,
21         "<=": lambda x, y: x <= y,
22         ">=": lambda x, y: x >= y
23     }
24
25     def __init__(self, policy_file_path: str = "data_collector/policies/
    policies_conf.json"):
26         """
27         Inizializza il PolicyManager.
28
29         Args:
30             policy_file_path: Percorso del file delle policy
31         """
32         self.policy_file_path = policy_file_path
33         self.policies = self._load_policies()
34         # Stato degli attuatori per ogni pianta {plant_id: {actuator_type:
    state}}
35         self.actuator_states = {}

```

```

36     logging.info(f"PolicyManager inizializzato con {len(self.policies)}
    piante")
37
38     def _load_policies(self) -> Dict[str, List[Dict]]:
39         """
40         Carica le policy dal file JSON.
41
42         Returns:
43             Dizionario con plant_id -> lista di policy
44         """
45         try:
46             if not os.path.exists(self.policy_file_path):
47                 logging.warning(f"File policy non trovato: {self.
policy_file_path}")
48                 return {}
49
50             with open(self.policy_file_path, 'r', encoding='utf-8') as f:
51                 data = json.load(f)
52
53             # Converte la struttura in dizionario
54             policies_dict = {}
55             for plant_policy in data:
56                 plant_id = plant_policy.get("plant_id")
57                 policies = plant_policy.get("policies", [])
58                 if plant_id:
59                     policies_dict[plant_id] = policies
60                 logging.info(f"Caricate {len(policies)} policy per {
plant_id}")
61
62             return policies_dict
63
64         except Exception as e:
65             logging.error(f"Errore caricamento policy: {e}")
66             return {}
67
68     def evaluate_policies(self, plant: PlantDescriptor, sensor_values: Dict
[str, float]) -> Tuple[List[str], List[str]]:
69         """
70         Valuta le policy per una pianta con i valori dei sensori.
71
72         Args:
73             plant: PlantDescriptor della pianta
74             sensor_values: Dizionario con i valori dei sensori {sensor_type
: value}
75
76         Returns:
77             Tuple (actions, alerts) - Lista di azioni e alert generati
78         """
79         plant_id = plant.plant_id
80         policies = self.policies.get(plant_id, [])
81
82         if not policies:
83             logging.debug(f"Nessuna policy trovata per {plant_id}")
84             return [], []
85
86         if plant_id not in self.actuator_states:
87             self.actuator_states[plant_id] = {}
88
89         actions = []
90         alerts = []
91
92         logging.info(f"Valutazione {len(policies)} policy per {plant_id}")
93
94         for policy in policies:

```

```

95         try:
96             sensor_type = policy.get("sensor")
97             condition = policy.get("condition")
98             threshold_value = policy.get("value")
99             action = policy.get("action")
100
101             if not all([sensor_type, condition, threshold_value, action
102 ]):
103                 logging.warning(f"Policy incompleta per {plant_id}: {
104 policy}")
105                 continue
106
107             if sensor_type not in sensor_values:
108                 logging.debug(f"Sensore {sensor_type} non disponibile
109 per {plant_id}")
110                 continue
111
112             sensor_value = sensor_values[sensor_type]
113
114             if condition not in self.OPERATORS:
115                 logging.warning(f"Operatore non valido: {condition}")
116                 continue
117
118             operator_func = self.OPERATORS[condition]
119             condition_met = operator_func(sensor_value, threshold_value
120 )
121
122             logging.debug(f"Policy {plant_id}: {sensor_type} {condition
123 } {threshold_value} -> {sensor_value} = {condition_met}")
124
125             if condition_met:
126                 if action == "alert":
127                     message = policy.get("message", f"Alert: {
128 sensor_type} {condition} {threshold_value}")
129                     alerts.append(message)
130                     logging.info(f"Alert generato per {plant_id}: {
131 message}")
132
133                 elif action in ["activate", "start", "on"]:
134                     actuator_type = policy.get("actuator", "irrigation"
135 )
136                     current_state = self.actuator_states[plant_id].get(
137 actuator_type, "off")
138
139                     if current_state != "on":
140                         action_str = f"Activate {actuator_type}"
141                         actions.append(action_str)
142                         self.actuator_states[plant_id][actuator_type] =
143 "on"
144
145                         logging.info(f"Azione generata per {plant_id}:
146 {action_str}")
147
148                     else:
149                         logging.debug(f"Attuatore {actuator_type} gi
150 attivo per {plant_id}")
151
152                     elif action in ["deactivate", "stop", "off"]:
153                         actuator_type = policy.get("actuator", "irrigation"
154 )
155                         current_state = self.actuator_states[plant_id].get(
156 actuator_type, "off")

```

```

145         if current_state != "off":
146             action_str = f"Deactivate {actuator_type}"
147             actions.append(action_str)
148             self.actuator_states[plant_id][actuator_type] =
149                 "off"
150             logging.info(f"Azione generata per {plant_id}:
151 {action_str}")
152         else:
153             logging.debug(f"Attuatore {actuator_type} gi
154 disattivo per {plant_id}")
155         else:
156             logging.warning(f"Azione non riconosciuta: {action}
157 ")
158     except Exception as e:
159         logging.error(f"Errore valutazione policy per {plant_id}: {
160 e}")
161         continue
162     return actions, alerts
163
164 def execute_actions(self, plant: PlantDescriptor, actions: List[str])
165 -> None:
166     """
167     Esegue le azioni generate dalle policy.
168
169     Args:
170         plant: PlantDescriptor della pianta
171         actions: Lista di azioni da eseguire
172     """
173     if not actions:
174         return
175     for action in actions:
176         try:
177             logging.info(f"Esecuzione azione per {plant.plant_id}: {
178 action}")
179
180             producer = DataCollectorProducer(plant, action)
181             producer.run()
182
183             logging.info(f"Azione eseguita: {action}")
184         except Exception as e:
185             logging.error(f"Errore esecuzione azione {action} per {
186 plant.plant_id}: {e}")
187
188 def get_policies_for_plant(self, plant_id: str) -> List[Dict]:
189     """
190     Restituisce le policy per una pianta specifica.
191
192     Args:
193         plant_id: ID della pianta
194
195     Returns:
196         Lista delle policy per la pianta
197     """
198     return self.policies.get(plant_id, [])
199
200 def reload_policies(self) -> None:
201     """Ricarica le policy dal file."""
202     self.policies = self._load_policies()
203     logging.info("Policy ricaricate")

```

A.3.6 json_manager.py

```

1 import json
2 import os
3 import time
4 from datetime import datetime
5 from typing import Dict, Any, List, Optional
6 import logging
7
8 class JsonManager:
9     """
10     Gestore per il salvataggio dei dati delle piante in file JSON.
11
12     Ogni pianta ha un file JSON separato nella cartella cloud_simulator/
13     plants_log/
14
15     """
16     def __init__(self, base_path: str = "cloud_simulator/plants_log"):
17         """
18         Inizializza il JsonManager.
19
20         Args:
21             base_path: Percorso base per i file JSON delle piante
22         """
23         self.base_path = base_path
24         self._ensure_directory_exists()
25
26     def _ensure_directory_exists(self):
27         """Assicura che la directory esista."""
28         if not os.path.exists(self.base_path):
29             os.makedirs(self.base_path)
30             logging.info(f"Creata directory: {self.base_path}")
31
32     def _get_plant_file_path(self, plant_id: str) -> str:
33         """Restituisce il percorso del file JSON per una pianta."""
34         return os.path.join(self.base_path, f"{plant_id}.json")
35
36     def _load_plant_data(self, plant_id: str) -> Dict[str, Any]:
37         """
38         Carica i dati di una pianta dal file JSON.
39
40         Args:
41             plant_id: ID della pianta
42
43         Returns:
44             Dizionario con i dati della pianta
45         """
46         file_path = self._get_plant_file_path(plant_id)
47
48         if os.path.exists(file_path):
49             try:
50                 with open(file_path, 'r', encoding='utf-8') as f:
51                     data = json.load(f)
52
53                 if isinstance(data, dict):
54                     return data
55                 else:
56                     logging.warning(f"Formato file non supportato in {
57 file_path}: {type(data)}")
58                     return self._create_empty_plant_data(plant_id)
59

```

```

60         except (json.JSONDecodeError, IOError) as e:
61             logging.warning(f"Errore caricamento file {file_path}: {e}")
62     )
63     return self._create_empty_plant_data(plant_id)
64 else:
65     return self._create_empty_plant_data(plant_id)
66
67 def _create_empty_plant_data(self, plant_id: str, species: str = "
unknown") -> Dict[str, Any]:
68     """
69     Crea una struttura dati vuota per una pianta.
70
71     Args:
72         plant_id: ID della pianta
73         species: Specie della pianta
74
75     Returns:
76         Struttura dati vuota
77     """
78     now = datetime.now()
79     return {
80         "plant_id": plant_id,
81         "species": species,
82         "created_at": now.isoformat() + "Z",
83         "last_updated": now.isoformat() + "Z",
84         "sensors": {},
85         "actuators": {}
86     }
87
88 def _save_plant_data(self, plant_id: str, data: Dict[str, Any]):
89     """
90     Salva i dati di una pianta nel file JSON.
91
92     Args:
93         plant_id: ID della pianta
94         data: Dati da salvare
95     """
96     file_path = self._get_plant_file_path(plant_id)
97
98     try:
99         # Aggiorna timestamp
100         data["last_updated"] = datetime.now().isoformat() + "Z"
101
102         with open(file_path, 'w', encoding='utf-8') as f:
103             json.dump(data, f, indent=2, ensure_ascii=False)
104
105         logging.debug(f"Dati salvati per {plant_id} in {file_path}")
106
107     except IOError as e:
108         logging.error(f"Errore salvataggio file {file_path}: {e}")
109         raise
110
111 def save_sensor_data(self, plant_id: str, sensor_type: str, value:
float,
112                     species: str = "unknown", max_entries: int = 1000):
113     """
114     Salva i dati di un sensore per una pianta.
115
116     Args:
117         plant_id: ID della pianta
118         sensor_type: Tipo di sensore (es. "humidity", "temperature")
119         value: Valore del sensore
120         species: Specie della pianta (per creazione file se non esiste)
121         max_entries: Numero massimo di entry da mantenere per sensore

```

```

121     """
122     try:
123         data = self._load_plant_data(plant_id)
124
125         if data["species"] == "unknown" and species != "unknown":
126             data["species"] = species
127
128         if sensor_type not in data["sensors"]:
129             data["sensors"][sensor_type] = []
130
131         timestamp = int(time.time())
132         entry = {
133             "value": value,
134             "timestamp": timestamp
135         }
136
137         data["sensors"][sensor_type].append(entry)
138
139         if len(data["sensors"][sensor_type]) > max_entries:
140             data["sensors"][sensor_type] = data["sensors"][sensor_type]
141             [-max_entries:]
142
143         self._save_plant_data(plant_id, data)
144
145         logging.info(f"Sensore {sensor_type} salvato per {plant_id}: {
146             value}")
147
148     except Exception as e:
149         logging.error(f"Errore salvataggio sensore {sensor_type} per {
150             plant_id}: {e}")
151         raise
152
153     def save_actuator_data(self, plant_id: str, actuator_type: str, action:
154         str,
155                             species: str = "unknown", max_entries: int =
156         1000):
157         """
158         Salva i dati di un attuatore per una pianta.
159
160         Args:
161             plant_id: ID della pianta
162             actuator_type: Tipo di attuatore (es. "irrigation")
163             action: Azione eseguita (es. "start", "stop")
164             species: Specie della pianta (per creazione file se non esiste)
165             max_entries: Numero massimo di entry da mantenere per attuatore
166         """
167         try:
168             data = self._load_plant_data(plant_id)
169
170             if data["species"] == "unknown" and species != "unknown":
171                 data["species"] = species
172
173             if actuator_type not in data["actuators"]:
174                 data["actuators"][actuator_type] = []
175
176             timestamp = int(time.time())
177             entry = {
178                 "action": action,
179                 "timestamp": timestamp
180             }
181
182             data["actuators"][actuator_type].append(entry)

```

```

180         if len(data["actuators"][actuator_type]) > max_entries:
181             data["actuators"][actuator_type] = data["actuators"][
182                 actuator_type][-max_entries:]
183
184         self._save_plant_data(plant_id, data)
185
186         logging.info(f"Attuatore {actuator_type} salvato per {plant_id}
187         ): {action}")
188
189         except Exception as e:
190             logging.error(f"Errore salvataggio attuatore {actuator_type}
191             per {plant_id}: {e}")
192             raise
193
194     def get_plant_data(self, plant_id: str) -> Optional[Dict[str, Any]]:
195         """
196         Recupera i dati di una pianta.
197
198         Args:
199             plant_id: ID della pianta
200
201         Returns:
202             Dati della pianta o None se non trovata
203         """
204         try:
205             return self._load_plant_data(plant_id)
206         except Exception as e:
207             logging.error(f"Errore recupero dati per {plant_id}: {e}")
208             return None
209
210     def list_plants(self) -> List[str]:
211         """
212         Lista tutte le piante con file JSON.
213
214         Returns:
215             Lista degli ID delle piante
216         """
217         try:
218             plants = []
219             for filename in os.listdir(self.base_path):
220                 if filename.endswith('.json'):
221                     plant_id = filename[:-5]
222                     plants.append(plant_id)
223             return plants
224         except Exception as e:
225             logging.error(f"Errore lista piante: {e}")
226             return []
227
228     def get_latest_sensor_value(self, plant_id: str, sensor_type: str) ->
229     Optional[float]:
230         """
231         Recupera l'ultimo valore di un sensore per una pianta.
232
233         Args:
234             plant_id: ID della pianta
235             sensor_type: Tipo di sensore
236
237         Returns:
238             Ultimo valore del sensore o None se non trovato
239         """
240         try:
241             data = self._load_plant_data(plant_id)

```

```

240         if sensor_type in data["sensors"] and data["sensors"][
sensor_type]:
241             return data["sensors"][sensor_type][-1]["value"]
242         return None
243     except Exception as e:
244         logging.error(f"Errore recupero ultimo valore sensore {
sensor_type} per {plant_id}: {e}")
245         return None
246
247     def get_sensor_history(self, plant_id: str, sensor_type: str, hours:
int = 24) -> List[Dict[str, Any]]:
248         """
249         Recupera la cronologia di un sensore per una pianta.
250
251         Args:
252             plant_id: ID della pianta
253             sensor_type: Tipo di sensore (es. "humidity", "temperature")
254             hours: Numero di ore di cronologia da recuperare
255
256         Returns:
257             Lista di dizionari con {timestamp, value, time_formatted}
258         """
259         try:
260             data = self._load_plant_data(plant_id)
261             if sensor_type not in data["sensors"]:
262                 return []
263
264             current_time = int(time.time())
265             cutoff_time = current_time - (hours * 3600)
266
267             sensor_data = data["sensors"][sensor_type]
268             filtered_data = [
269                 entry for entry in sensor_data
270                 if entry["timestamp"] >= cutoff_time
271             ]
272
273             formatted_data = []
274             for entry in filtered_data:
275                 timestamp = entry["timestamp"]
276                 value = entry["value"]
277
278                 dt = datetime.fromtimestamp(timestamp)
279                 time_formatted = dt.strftime("%H:%M")
280
281                 formatted_data.append({
282                     "timestamp": timestamp,
283                     "value": value,
284                     "time_formatted": time_formatted,
285                     "hour": dt.hour,
286                     "minute": dt.minute
287                 })
288
289             formatted_data.sort(key=lambda x: x["timestamp"])
290
291             logging.info(f"Recuperati {len(formatted_data)} punti per {
sensor_type} di {plant_id}")
292             return formatted_data
293
294         except Exception as e:
295             logging.error(f"Errore recupero cronologia sensore {sensor_type
} per {plant_id}: {e}")
296             return []
297
298     def process_sensor_data_and_evaluate_policies(self, plant_descriptor,

```

```

299     sensor_type: str, sensor_value: float, policy_manager):
300         """
301         Processa i dati del sensore, li salva e valuta le policy.
302
303         Args:
304             plant_descriptor: PlantDescriptor della pianta
305             sensor_type: Tipo di sensore (es. "humidity", "temperature")
306             sensor_value: Valore del sensore ricevuto
307             policy_manager: PolicyManager per la valutazione delle policy
308
309         Returns:
310             List[str]: Lista delle azioni eseguite
311         """
312         try:
313             self.save_sensor_data(
314                 plant_id=plant_descriptor.plant_id,
315                 sensor_type=sensor_type,
316                 value=sensor_value,
317                 species=plant_descriptor.species
318             )
319             print(f"        Dati sensore salvati: {sensor_type} = {
320                 sensor_value}")
321
322             for sensor in plant_descriptor.sensors:
323                 if sensor.type == sensor_type:
324                     sensor.value = sensor_value
325                     break
326
327             sensor_values = {}
328             for sensor in plant_descriptor.sensors:
329                 sensor_values[sensor.type] = sensor.value
330
331             actions, alerts = policy_manager.evaluate_policies(
332                 plant_descriptor, sensor_values)
333
334             for alert in alerts:
335                 print(f"        ALERT per {plant_descriptor.plant_id}: {alert}")
336
337             return actions
338
339         except Exception as e:
340             logging.error(f"Errore processamento dati sensore {sensor_type
341                 }: {e}")
342             return []

```

A.4 Backend

A.4.1 web_api_server.py

```

1 import os
2 import sys
3
4 project_root = os.path.abspath(os.path.join(os.path.dirname(__file__), '..'
5     ))
6 sys.path.insert(0, project_root)
7
8 import json
9 import time

```

```

9 from datetime import datetime, timedelta
10 from typing import Dict, List, Any, Optional
11 from flask import Flask, jsonify, request
12 from flask_cors import CORS
13 import logging
14 from data_collector.data_collector_producer import DataCollectorProducer
15 from data_collector.plant_descriptor import PlantDescriptor
16 from data_collector.json_manager import JsonManager
17
18 logging.basicConfig(level=logging.INFO)
19 logger = logging.getLogger(__name__)
20
21 app = Flask(__name__)
22 CORS(app)
23
24 PLANTS_LOG_PATH = "cloud_simulator/plants_log"
25 DEFAULT_POLLING_INTERVAL = 5 # secondi
26
27 class PlantDataService:
28     """Servizio per leggere e processare i dati delle piante usando
29     JsonManager."""
30
31     def __init__(self, plants_log_path: str):
32         self.json_manager = JsonManager(base_path=plants_log_path)
33
34     def get_all_plants_data(self) -> List[Dict[str, Any]]:
35         """Legge tutti i dati delle piante usando JsonManager."""
36         plants_data = []
37
38         try:
39             plant_ids = self.json_manager.list_plants()
40
41             for plant_id in plant_ids:
42                 plant_data = self._load_plant_data(plant_id)
43                 if plant_data:
44                     plants_data.append(plant_data)
45
46             logger.info(f"Caricati dati per {len(plants_data)} piante")
47             return plants_data
48
49         except Exception as e:
50             logger.error(f"Errore nel caricamento dati piante: {e}")
51             return []
52
53     def _load_plant_data(self, plant_id: str) -> Optional[Dict[str, Any]]:
54         """Carica i dati di una singola pianta usando JsonManager."""
55         try:
56             raw_data = self.json_manager.get_plant_data(plant_id)
57
58             if not raw_data:
59                 logger.warning(f"Pianta {plant_id} non trovata")
60                 return None
61
62             return self._process_plant_data_for_frontend(plant_id, raw_data)
63
64         except Exception as e:
65             logger.error(f"Errore nel caricamento pianta {plant_id}: {e}")
66             return None
67
68     def _process_plant_data_for_frontend(self, plant_id: str, data: Dict[

```

```

71     str, Any]) -> Dict[str, Any]:
72         """Processa i dati della pianta per il formato richiesto dal
73         frontend."""
74         if not isinstance(data, dict):
75             logger.warning(f"Formato dati non supportato per {plant_id}: {
76             type(data)}")
77             return None
78
79         species = data.get("species", "unknown")
80         last_updated = data.get("last_updated", "")
81         sensors_data = data.get("sensors", {})
82         actuators_data = data.get("actuators", {})
83
84         soil_moisture_percentage = self._get_latest_sensor_value(
85             sensors_data, "humidity", 50)
86
87         is_watering = self._is_irrigation_active(actuators_data)
88
89         last_watered = self._get_last_watering_time(actuators_data)
90
91         return {
92             "id": plant_id,
93             "name": f"{plant_id}",
94             "type": species,
95             "soilMoisture": soil_moisture_percentage,
96             "isWatering": is_watering,
97             "lastWatered": last_watered,
98             "lastUpdated": last_updated,
99             "sensors": self._format_sensors_data(sensors_data),
100             "actuators": self._format_actuators_data(actuators_data)
101         }
102
103     def _get_latest_sensor_value(self, sensors_data: Dict, sensor_type: str
104     , default_value: float) -> float:
105         """Ottiene l'ultimo valore di un sensore."""
106         sensor_entries = sensors_data.get(sensor_type, [])
107         if sensor_entries and len(sensor_entries) > 0:
108             return float(sensor_entries[-1].get("value", default_value))
109         return default_value
110
111     def _is_irrigation_active(self, actuators_data: Dict) -> bool:
112         """Determina se l'irrigazione attualmente attiva."""
113         irrigation_entries = actuators_data.get("irrigation", [])
114         if irrigation_entries and len(irrigation_entries) > 0:
115             last_action = irrigation_entries[-1].get("action", "")
116             return last_action.lower() in ["start", "on", "activate"]
117         return False
118
119     def _get_last_watering_time(self, actuators_data: Dict) -> str:
120         """Calcola il tempo dell'ultima irrigazione."""
121         irrigation_entries = actuators_data.get("irrigation", [])
122         if irrigation_entries and len(irrigation_entries) > 0:
123             last_timestamp = irrigation_entries[-1].get("timestamp", 0)
124             if last_timestamp:
125                 last_time = datetime.fromtimestamp(last_timestamp)
126                 now = datetime.now()
127                 diff = now - last_time
128
129                 if diff.total_seconds() < 60:
130                     return f"{int(diff.total_seconds())} secondi fa"
131                 elif diff.total_seconds() < 3600:
132                     return f"{int(diff.total_seconds() / 60)} minuti fa"

```

```

130         else:
131             return f"{int(diff.total_seconds() / 3600)} ore fa"
132
133     return "Mai"
134
135     def _format_sensors_data(self, sensors_data: Dict) -> Dict[str, Any]:
136         """Formatta i dati dei sensori per il frontend."""
137         formatted = {}
138         for sensor_type, entries in sensors_data.items():
139             if entries and len(entries) > 0:
140                 latest_entry = entries[-1]
141                 formatted[sensor_type] = {
142                     "value": latest_entry.get("value", 0),
143                     "timestamp": latest_entry.get("timestamp", 0),
144                     "history": entries[-10:] if len(entries) > 10 else
entries # Ultimi 10 valori
145                 }
146             return formatted
147
148     def _format_actuators_data(self, actuators_data: Dict) -> Dict[str, Any
]:
149         """Formatta i dati degli attuatori per il frontend."""
150         formatted = {}
151         for actuator_type, entries in actuators_data.items():
152             if entries and len(entries) > 0:
153                 latest_entry = entries[-1]
154                 formatted[actuator_type] = {
155                     "action": latest_entry.get("action", ""),
156                     "timestamp": latest_entry.get("timestamp", 0),
157                     "history": entries[-10:] if len(entries) > 10 else
entries # Ultimi 10 valori
158                 }
159             return formatted
160
161     def _send_mqtt_command(self, plant_id: str, actuator_type: str, command
: str) -> bool:
162         """Invia un comando MQTT all'attuatore usando DataCollectorProducer
163         ."""
164         try:
165             plant_descriptor = PlantDescriptor(species="unknown", plant_id=
plant_id)
166
167             producer = DataCollectorProducer(
168                 plant_descriptor=plant_descriptor,
169                 command=f"{command} {actuator_type}",
170                 json_path=self.json_manager.base_path
171             )
172
173             producer.run()
174
175             logger.info(f"Comando MQTT inviato tramite
DataCollectorProducer: {actuator_type} = {command}")
176             return True
177
178         except Exception as e:
179             logger.error(f"Errore invio comando MQTT {actuator_type}: {e}")
180             return False
181
182     def _format_actuators_data(self, actuators_data: Dict) -> Dict[str, Any
]:
183         """Formatta i dati degli attuatori per il frontend."""
184         formatted = {}
185         for actuator_type, entries in actuators_data.items():

```

```

186         if entries and len(entries) > 0:
187             latest_entry = entries[-1]
188             formatted[actuator_type] = {
189                 "action": latest_entry.get("action", ""),
190                 "timestamp": latest_entry.get("timestamp", 0),
191                 "history": entries[-10:] if len(entries) > 10 else
entries # Ultimi 10 valori
192             }
193             return formatted
194
195 def convert_action_to_simple_command(action: str) -> str:
196     """Converte un'azione complessa in comando semplice."""
197     action_lower = action.lower()
198
199     if action_lower in ['deactivate', 'stop', 'off']:
200         return 'stop'
201     elif action_lower in ['activate', 'start', 'on']:
202         return 'start'
203     else:
204         return action_lower
205
206
207 plant_service = PlantDataService(PLANTS_LOG_PATH)
208
209 @app.route('/api/plants', methods=['GET'])
210 def get_all_plants():
211     """Endpoint per ottenere tutti i dati delle piante."""
212     try:
213         plants_data = plant_service.get_all_plants_data()
214
215         # Se non ci sono dati, restituisci dati di esempio
216         if not plants_data:
217             logger.warning("Nessun dato pianta trovato, restituisco dati di
esempio")
218             plants_data = [
219                 {
220                     "id": "my_pomodoro",
221                     "name": "Pomodoro (my_pomodoro)",
222                     "type": "pomodoro",
223                     "soilMoisture": 75,
224                     "isWatering": False,
225                     "lastWatered": "Nessun dato disponibile",
226                     "lastUpdated": datetime.now().isoformat(),
227                     "sensors": {},
228                     "actuators": {}
229                 }
230             ]
231
232             return jsonify(plants_data)
233
234     except Exception as e:
235         logger.error(f"Errore nell'endpoint /api/plants: {e}")
236         return jsonify({"error": str(e)}), 500
237
238 @app.route('/api/plants/<plant_id>', methods=['GET'])
239 def get_plant_by_id(plant_id: str):
240     """Endpoint per ottenere i dati di una singola pianta."""
241     try:
242         plant_data = plant_service._load_plant_data(plant_id)
243
244         if not plant_data:
245             return jsonify({"error": f"Pianta {plant_id} non trovata"}),
404
246

```

```

247         return jsonify(plant_data)
248
249     except Exception as e:
250         logger.error(f"Errore nell'endpoint /api/plants/{plant_id}: {e}")
251         return jsonify({"error": str(e)}), 500
252
253 @app.route('/api/plants/<plant_id>/actuators/<actuator_type>', methods=['
    POST'])
254 def control_actuator(plant_id: str, actuator_type: str):
255     """Endpoint per controllare manualmente un attuatore."""
256     try:
257         # Leggi il payload della richiesta
258         data = request.get_json()
259         if not data or 'action' not in data:
260             return jsonify({"error": "Payload mancante o formato non valido
    "}), 400
261
262         action = data['action'].lower()
263
264         # Valida l'azione
265         valid_actions = ['start', 'stop', 'on', 'off', 'activate', '
    deactivate']
266         if action not in valid_actions:
267             return jsonify({"error": f"Azione non valida. Valori accettati:
    {valid_actions}"}), 400
268
269         # Verifica che la pianta esista
270         plant_data = plant_service._load_plant_data(plant_id)
271         if not plant_data:
272             return jsonify({"error": f"Pianta {plant_id} non trovata"}),
    404
273
274         # Converti l'azione in comando semplice
275         simple_command = convert_action_to_simple_command(action)
276
277         # Invia comando MQTT all'attuatore (che salva anche nel JSON)
278         mqtt_success = plant_service._send_mqtt_command(plant_id,
    actuator_type, simple_command)
279
280         return jsonify({
281             "success": True,
282             "message": f"Comando {simple_command} inviato all'attuatore {
    actuator_type}",
283             "plant_id": plant_id,
284             "actuator_type": actuator_type,
285             "action": simple_command,
286             "mqtt_sent": mqtt_success,
287             "timestamp": datetime.now().isoformat()
288         })
289
290     except Exception as e:
291         logger.error(f"Errore nell'endpoint POST /api/plants/{plant_id}/
    actuators/{actuator_type}: {e}")
292         return jsonify({"error": str(e)}), 500
293
294 @app.route('/api/plants/<plant_id>/sensors/<sensor_type>/history', methods
    =['GET'])
295 def get_sensor_history(plant_id: str, sensor_type: str):
296     """Endpoint per ottenere la cronologia di un sensore."""
297     try:
298         hours = request.args.get('hours', 24, type=int)
299
300         logger.info(f"Richiesta cronologia {sensor_type} per {plant_id} (
    ultime {hours}h)")

```

```

301
302     history_data = plant_service.json_manager.get_sensor_history(
303         plant_id, sensor_type, hours)
304
305     return jsonify({
306         "success": True,
307         "plant_id": plant_id,
308         "sensor_type": sensor_type,
309         "hours": hours,
310         "data_points": len(history_data),
311         "data": history_data
312     })
313
314 except Exception as e:
315     logger.error(f"Errore recupero cronologia {sensor_type} per {
316         plant_id}: {e}")
317     return jsonify({"error": str(e)}), 500
318
319 @app.route('/api/health', methods=['GET'])
320 def health_check():
321     """Endpoint per il controllo dello stato del server."""
322     return jsonify({
323         "status": "healthy",
324         "timestamp": datetime.now().isoformat(),
325         "plants_log_path": PLANTS_LOG_PATH,
326         "plants_count": len(plant_service.get_all_plants_data())
327     })
328
329 @app.route('/api/stats', methods=['GET'])
330 def get_stats():
331     """Endpoint per ottenere statistiche del sistema."""
332     try:
333         plants_data = plant_service.get_all_plants_data()
334
335         stats = {
336             "total_plants": len(plants_data),
337             "active_irrigation": sum(1 for plant in plants_data if plant.
338                 get("isWatering", False)),
339             "avg_soil_moisture": sum(plant.get("soilMoisture", 0) for plant
340                 in plants_data) / len(plants_data) if plants_data else 0,
341             "last_update": datetime.now().isoformat()
342         }
343
344         return jsonify(stats)
345
346 except Exception as e:
347     logger.error(f"Errore nell'endpoint /api/stats: {e}")
348     return jsonify({"error": str(e)}), 500
349
350 if __name__ == "__main__":
351     print("    Avvio Web API Server...")
352     print(f"    Plants log path: {PLANTS_LOG_PATH}")
353     print(f"    API Base URL: http://localhost:5000/api")
354     print(f"    Endpoints disponibili:")
355     print(f"    - GET /api/plants - Tutte le piante")
356     print(f"    - GET /api/plants/<id> - Pianta specifica")
357     print(f"    - GET /api/health - Stato server")
358     print(f"    - GET /api/stats - Statistiche sistema")
359     print()
360
361     # Verifica che la directory esista
362     if not os.path.exists(PLANTS_LOG_PATH):
363         print(f"    Directory {PLANTS_LOG_PATH} non trovata!")
364         print("    Assicurati che il data_collector sia in esecuzione.")

```

```
361     print()
362
363     app.run(host='0.0.0.0', port=5000, debug=True)
```

A.5 Frontend

A.5.1 page.tsx

```
1  "use client"
2  import { useState, useEffect } from "react"
3  import { Leaf } from "lucide-react"
4  import { PlantDashboard } from "@components/plant-dashboard"
5  import { apiService, type Plant } from "@lib/api"
6
7  export default function SmartPlantDashboard() {
8    const [plants, setPlantsData] = useState<Plant[]>([])
9    const [loading, setLoading] = useState(true)
10   const [error, setError] = useState<string | null>(null)
11
12   const loadPlantsData = async () => {
13     try {
14       setLoading(true)
15       const plantsData = await apiService.getAllPlants()
16       setPlantsData(plantsData)
17       setError(null)
18     } catch (err) {
19       console.error("Failed to load plants data:", err)
20       setError("Errore nel caricamento dei dati delle piante")
21       setPlantsData([
22         {
23           id: "1",
24           name: "Basilico",
25           type: "Erba aromatica",
26           soilMoisture: 65,
27           isWatering: false,
28           lastWatered: "2 ore fa",
29         },
30         {
31           id: "2",
32           name: "Pomodoro",
33           type: "Ortaggio",
34           soilMoisture: 45,
35           isWatering: true,
36           lastWatered: "30 minuti fa",
37         },
38       ])
39     } finally {
40       setLoading(false)
41     }
42   }
43
44   const handleIrrigationControl = async (plantId: string, action: "start" |
45     "stop") => {
46     console.log(`      Irrigation control: ${plantId} - ${action}`)
47     try {
48       console.log(`      Calling API...`)
49       await apiService.controlIrrigation(plantId, action)
50       console.log(`      API call successful, reloading data...`)
51       // Ricarica i dati per aggiornare lo stato
```

```

51     await loadPlantsData()
52     console.log('    Data reloaded')
53   } catch (err) {
54     console.error("    Failed to control irrigation:", err)
55     setError("Errore nel controllo dell'irrigazione")
56   }
57 }
58
59 useEffect(() => {
60   loadPlantsData()
61
62   // Poll for updates every 10 seconds
63   const interval = setInterval(() => {
64     loadPlantsData()
65   }, 10000)
66
67   return () => clearInterval(interval)
68 }, [])
69
70 if (loading) {
71   return (
72     <div className="min-h-screen bg-gradient-to-br from-green-50 to-blue-50 dark:from-gray-900 dark:to-gray-800 flex items-center justify-center p-4">
73       <div className="text-center">
74         <Leaf className="h-12 w-12 text-green-500 animate-pulse mx-auto mb-4" />
75         <p className="text-lg text-gray-600 dark:text-gray-300">
76           Caricamento dati delle piante...</p>
77       </div>
78     )
79   }
80
81   return (
82     <div className="min-h-screen bg-gradient-to-br from-green-50 to-blue-50 dark:from-gray-900 dark:to-gray-800">
83       <div className="container mx-auto p-4 sm:p-6 lg:p-8">
84         {/* Header */}
85         <div className="mb-6 sm:mb-8">
86           <div className="flex justify-center mb-4">
87             <div className="flex flex-col sm:flex-row items-center gap-3 text-center sm:text-left">
88               <div className="p-2 bg-green-500 rounded-lg flex-shrink-0">
89                 <Leaf className="h-6 w-6 text-white" />
90               </div>
91               <div>
92                 <h1 className="text-2xl sm:text-3xl lg:text-4xl font-bold text-gray-900 dark:text-white">
93                   Smart Plants - IoT System
94                 </h1>
95                 <p className="text-sm sm:text-base text-gray-600 dark:text-gray-300 mt-1">
96                   Sistema intelligente per la gestione e monitoraggio dei tuoi vasi
97                 </p>
98                 {error && <p className="text-red-500 text-xs sm:text-sm mt-1">{error}</p>} - Usando dati di fallback</p>
99               </div>
100            </div>
101          </div>
102        </div>
103      </div>
104      {/* Main Content - Only Dashboard */}

```

```

105     <PlantDashboard
106       plants={plants}
107       setPlantsData={setPlantsData}
108       onIrrigationControl={handleIrrigationControl}
109     />
110   </div>
111 </div>
112 )
113 }

```

A.5.2 plant-dashboard.tsx

```

1  "use client"
2
3  import type React from "react"
4
5  import { Card, CardContent, CardDescription, CardHeader, CardTitle } from "
    @/components/ui/card"
6  import { Button } from "@/components/ui/button"
7  import { Progress } from "@/components/ui/progress"
8  import { Droplets, Leaf, Play, Pause, ChevronDown, ChevronUp } from "lucide
    -react"
9  import Image from "next/image"
10 import { useState, useEffect } from "react"
11 import { LineChart, Line, XAxis, YAxis, CartesianGrid, Tooltip,
    ResponsiveContainer } from 'recharts'
12
13 interface Plant {
14   id: string
15   name: string
16   type: string
17   soilMoisture: number
18   isWatering: boolean
19   lastWatered: string
20 }
21
22 interface PlantDashboardProps {
23   plants: Plant[]
24   setPlantsData: (plants: Plant[]) => void
25   onIrrigationControl?: (plantId: string, action: "start" | "stop") => void
26 }
27
28 interface PlantCardProps {
29   plant: Plant
30   onIrrigationControl?: (plantId: string, action: "start" | "stop") => void
31 }
32
33 // Componente per gestire le immagini delle piante con fallback
34 function PlantImage({ plantType, plantName }: { plantType: string;
    plantName: string }) {
35   const [currentImage, setCurrentImage] = useState<string>('')
36   const [imageError, setImageError] = useState(false)
37
38   // Funzione per ottenere l'immagine della pianta basata sul tipo
39   const getPlantImage = (plantType: string): string => {
40     const type = plantType.toLowerCase()
41
42     // Cerca prima PNG, poi JPG come fallback
43     return `/plant-images/${type}.png`
44   }

```

```

45 // Inizializza l'immagine corrente
46 useEffect(() => {
47   setCurrentImage(getPlantImage(plantType))
48   setImageError(false)
49 }, [plantType])
50
51
52 const handleImageError = () => {
53   if (!imageError) {
54     setImageError(true)
55     // Prova fallback con PNG se JPG fallisce
56     if (currentImage.endsWith('.jpg')) {
57       setCurrentImage(currentImage.replace('.jpg', '.png'))
58     } else if (currentImage.endsWith('.png')) {
59       // Se anche PNG fallisce, usa immagine generica
60       setCurrentImage('/plant-images/plant.jpg')
61     }
62   }
63 }
64
65 return (
66   <div className="w-24 h-24 sm:w-28 sm:h-28 md:w-32 md:h-32 lg:w-36 lg:h-36 xl:w-40 xl:h-40 relative">
67     <Image
68       src={currentImage}
69       alt={`${plantName} - ${plantType}`}
70       fill
71       className="object-contain rounded-lg"
72       onError={handleImageError}
73       priority={true}
74     />
75   </div>
76 )
77 }
78
79 // Componente per il grafico umidit integrato
80 function HumidityChart({ plantId }: { plantId: string }) {
81   const [data, setData] = useState<any[]>([])
82   const [loading, setLoading] = useState(true)
83
84   useEffect(() => {
85     const fetchData = async () => {
86       try {
87         const response = await fetch('http://localhost:5000/api/plants/${plantId}/sensors/humidity/history?hours=24')
88         if (response.ok) {
89           const result = await response.json()
90           if (result.success && result.data) {
91             const chartData = result.data.map((point: any) => ({
92               time: point.time_formatted,
93               humidity: point.value,
94               timestamp: point.timestamp
95             }))
96             setData(chartData)
97           }
98         }
99       } catch (error) {
100         console.error('Errore fetch dati umidit :', error)
101       } finally {
102         setLoading(false)
103       }
104     }
105
106     fetchData()

```

```

107   }, [plantId])
108
109   if (loading) {
110     return (
111       <div className="h-32 flex items-center justify-center">
112         <div className="text-gray-500 text-sm">Caricamento...</div>
113       </div>
114     )
115   }
116
117   if (data.length === 0) {
118     return (
119       <div className="h-32 flex items-center justify-center">
120         <div className="text-gray-500 text-sm">Nessun dato disponibile</div>
121       </div>
122     )
123   }
124
125   return (
126     <div className="h-20 sm:h-24 md:h-28 lg:h-32">
127       <ResponsiveContainer width="100%" height="100%">
128         <LineChart data={data}>
129           <CartesianGrid strokeDasharray="3 3" />
130           <XAxis
131             dataKey="time"
132             tick={{ fontSize: 8 }}
133             interval="preserveStartEnd"
134           />
135           <YAxis
136             domain={[0, 100]}
137             tick={{ fontSize: 8 }}
138           />
139           <Tooltip
140             formatter={(value: any) => ['${value.toFixed(1)}%', 'Umidit
141           ']}
142             labelFormatter={(label) => 'Ora: ${label}{'
143             contentStyle={{
144               backgroundColor: 'white',
145               border: '1px solid #ccc',
146               borderRadius: '4px',
147               fontSize: '10px'
148             }}
149           />
150           <Line
151             type="monotone"
152             dataKey="humidity"
153             stroke="#3b82f6"
154             strokeWidth={1.5}
155             dot={false}
156           />
157         </LineChart>
158       </ResponsiveContainer>
159     </div>
160   )
161 }
162
163 // Componente per la singola card pianta
164 function PlantCard({ plant, onIrrigationControl }: PlantCardProps) {
165   const [showChart, setShowChart] = useState(false)
166
167   const handleManualControl = (plantId: string, action: "start" | "stop")
168     => {
169     console.log('      Manual control clicked: ${plantId} - ${action}')

```

```

168   if (onIrrigationControl) {
169     console.log('      Calling onIrrigationControl...')
170     onIrrigationControl(plantId, action)
171   } else {
172     console.log('      onIrrigationControl not provided')
173   }
174 }
175
176 const getMoistureColor = (value: number) => {
177   if (value < 30) return "bg-red-500"
178   if (value < 50) return "bg-yellow-500"
179   return "bg-green-500"
180 }
181
182 return (
183   <Card className="overflow-hidden w-full h-full flex flex-col">
184     <CardHeader className="pb-3 flex-shrink-0">
185       <div className="flex items-center gap-3">
186         {/* Immagine della pianta a sinistra */}
187         <div className="flex-shrink-0">
188           <PlantImage plantType={plant.type} plantName={plant.name} />
189         </div>
190
191         {/* Nome e specie a destra */}
192         <div className="min-w-0 flex-1">
193           <CardTitle className="text-sm sm:text-base lg:text-lg truncate
194             ">{plant.name}</CardTitle>
195           <CardDescription className="text-xs sm:text-sm truncate">{plant
196             .type}</CardDescription>
197         </div>
198       </div>
199     </CardHeader>
200
201     <CardContent className="space-y-2 sm:space-y-3 flex-1 flex flex-col">
202       <div className="flex items-center justify-between p-2 sm:p-3 bg-
203         blue-50 dark:bg-blue-950 rounded-lg">
204         <div className="flex items-center gap-2 min-w-0 flex-1">
205           <Droplets className="h-3 w-3 sm:h-4 sm:w-4 text-blue-600 flex-
206             shrink-0" />
207           <span className="text-xs sm:text-sm font-medium">Irrigazione</
208             span>
209           {plant.isWatering && <span className="text-xs text-blue-600
210             font-medium">ATTIVA</span>}
211         </div>
212         <div className="flex items-center gap-1 sm:gap-2 flex-shrink-0">
213           <Button
214             size="sm"
215             variant={plant.isWatering ? "destructive" : "default"}
216             onClick={() => handleManualControl(plant.id, plant.isWatering
217               ? "stop" : "start")}
218             className="h-6 w-6 sm:h-7 sm:w-7 lg:h-8 lg:w-8 p-0"
219             >
220             {plant.isWatering ? <Pause className="h-2 w-2 sm:h-3 sm:w-3"
221               /> : <Play className="h-2 w-2 sm:h-3 sm:w-3" />}
222           </Button>
223         </div>
224       </div>
225
226       <div className="space-y-1 sm:space-y-2">
227         <div className="flex items-center gap-2">
228           <Droplets className="h-3 w-3 sm:h-4 sm:w-4 text-cyan-500 flex-
229             shrink-0" />
230           <span className="text-xs sm:text-sm font-medium">Umidit </span
231         >

```

```

222     </div>
223     <Progress
224       value={plant.soilMoisture}
225       className="h-2 sm:h-3"
226       style={
227         {
228           "--progress-background": getMoistureColor(plant.
soilMoisture),
229         } as React.CSSProperties
230       }
231     />
232     <p className="text-xs sm:text-sm text-gray-600 dark:text-gray
-400">{plant.soilMoisture.toFixed(1)}%</p>
233   </div>
234
235   { /* Grafico umidit integrato nella card */ }
236   <div className="space-y-1 sm:space-y-2 flex-1">
237     <div className="flex items-center justify-between">
238       <div className="flex items-center gap-2">
239         <Droplets className="h-3 w-3 sm:h-4 sm:w-4 text-blue-500 flex
-shrink-0" />
240         <span className="text-xs sm:text-sm font-medium">Andamento
Umidit </span>
241       </div>
242       <Button
243         variant="ghost"
244         size="sm"
245         className="h-5 w-5 sm:h-6 sm:w-6 p-0"
246         onClick={() => setShowChart(!showChart)}
247       >
248         {showChart ? <ChevronUp className="h-3 w-3 sm:h-4 sm:w-4" />
: <ChevronDown className="h-3 w-3 sm:h-4 sm:w-4" />}
249       </Button>
250     </div>
251     {showChart && <HumidityChart plantId={plant.id} />}
252   </div>
253
254   <div className="text-xs text-gray-600 dark:text-gray-400 text-
center pt-1 sm:pt-2 border-t flex-shrink-0">
255     Ultima irrigazione: {plant.lastWatered}
256   </div>
257 </CardContent>
258 </Card>
259 )
260 }
261
262 export function PlantDashboard({ plants, setPlantsData, onIrrigationControl
}: PlantDashboardProps) {
263   const getGridClasses = () => {
264     if (plants.length === 1) {
265       return "flex justify-center max-w-sm mx-auto"
266     }
267     if (plants.length === 2) {
268       return "grid grid-cols-1 sm:grid-cols-2 gap-3 sm:gap-4 max-w-4xl mx-
auto"
269     }
270     return "grid grid-cols-1 sm:grid-cols-2 lg:grid-cols-3 xl:grid-cols-4
gap-3 sm:gap-4 lg:gap-6"
271   }
272
273   return (
274     <div className={getGridClasses()}>
275       {plants.map((plant) => (
276         <PlantCard

```

```
277         key={plant.id}
278         plant={plant}
279         onIrrigationControl={onIrrigationControl}
280     />
281   )}
282 </div>
283 )
284 }
```

Bibliografia

- [1] Arduino. *Arduino UNO R4 WiFi Datasheet*, 2025.
- [2] Ken Borgendale Andrew Banks, Eve Briggs and Rahul Gupta. *MQTT Version 5.0*, 2019.
- [3] Eclipse Mosquitto. *Mosquitto Documentation*, 2025.
- [4] Docker Inc. *Docker Documentation*, 2025.
- [5] Prakash Ghimire et al. *Comparative Study on Python Web Frameworks: Flask and Django*. PhD thesis, South-Eastern Finland University of Applied Sciences, 2020.
- [6] R. Morchid et al. Iot-enabled fire detection for sustainable agriculture: A real-time system using flask and embedded technologies. *Journal of King Saud University - Computer and Information Sciences*, 2024.
- [7] P. Bourhis. Json: Data model and query languages. *ScienceDirect*, 2020.
- [8] G. Langdale. Parsing gigabytes of json per second. *arXiv*, 2019.
- [9] M. Coblenz. A qualitative study of rest api design and specification. *VLHCC*, 2023.
- [10] G. Psaila. J-co: A platform-independent framework for managing json data. *MDPI*, 2021.
- [11] S. Bharany. Efficient middleware for the portability of paas services. *PMC*, 2022.
- [12] Microsoft. *The New TypeScript Handbook*, 2024.
- [13] Meta Platforms, Inc. *React Compiler Release Candidate*, 2025.
- [14] Vercel. *Next.js Documentation, Version 15*, 2025.
- [15] Recharts Team. *Recharts Official Documentation*, 2025.